Modern Simulation and Modeling

REUVEN Y. RUBINSTEIN
Technion, Israel Institute of Technology
Haifa, Israel

BENJAMIN MELAMED
Rutgers University
New Brunswick, New Jersey

A Wiley-Interscience Publication
JOHN WILEY & SONS, INC.
New York · Chichester · Weinheim · Brisbane · Singapore · Toronto

PART I

Conventional Simulation

Monte Carlo simulation (simulation, for short) came to the fore in the first few decades following World War II. The flowering of simulation as a theoretical discipline and practical tool manifested itself in a concomitant growth of the corresponding research and practitioner community. Today, simulation is in widespread use in countless academic institutions and industrial sites, and it has become a standard topic of teaching and research, with its own dedicated scientific journals and professional conferences.

Just as probability was historically motivated by gambling (Cardano and Pascal are credited with pioneering contributions (Feller, 1958, Vol. I, Section II.10)), the origins of simulation appear to lie in experiments involving chance (e.g., Buffon's needle experiment and other anecdotal evidence in Feller (1958, Vol. II, Section II.8). As such, simulation introduced an alternative paradigm to traditional analytical modeling with random elements. In both cases a well-defined model is presented, but the "solution method" is drastically different. The analytical approach employs strictly mathematical tools to compute various quantities of interest in relatively simple models. In contrast, the simulation approach merely generates possible histories (sample path realizations) and then calculates statistics from them. For example, to compute performance measures of a queueing system in equilibrium, such as the queue length distribution, the analytical approach calls for the solution of certain equilibrium equations, whereas the simulation approach generates sufficiently long queueing histories, and calculates the corresponding time averages. For the two approaches to lead to consistent results in this particular example, one relies on the presumed ergodicity of the system under study to ensure that time averages converge to the corresponding "phase averages." The different tools employed by the two approaches (mathematics versus statistics) are the source of their corresponding strengths and weaknesses: Analytical methods yield *exact* solutions but typically can handle only relatively simple models, whereas simulation can be applied to far more complex models but only yields statistical *estimates*, subject to experimental error. Nevertheless, therein lies the great advantage of simulation, namely, the fact that sample path realizations can always be computed, in principle, though subject to the physical constraints of computing time and available storage. Thus, a simulation environment serves as an in vitro laboratory that can afford the analyst enormous flexibility.

From a practical point of view, simulation has nowadays become the tool of choice for the analysis of most complex systems. The advent of powerful computer systems and graphical displays has provided the enabling technology for the explosive growth and dissemination of simulation technologies. Furthermore, the diffusion of simulation technology to a broad range of technology areas, such as manufacturing, telecommunications, and computer systems, has been accompanied by vigorous efforts to develop both general-purpose and special-purpose simulation languages and software environments.

This book is wholly devoted to discrete-event simulation—the most widespread paradigm of the simulation discipline. It covers probabilistic and statistical aspects of simulation; implementation issues, such as programming languages or parallel/distributed simulation, are outside its scope. The book is divided into

two parts: Part I treats *conventional simulation*, while Part II covers more recent developments that we term *modern simulation*. Roughly speaking, conventional simulation pertains to the fundamentals of discrete-event simulation, largely established before the 1980s, while modern simulation pertains to "smart techniques" dealing with sensitivity analysis and stochastic optimization of computer simulation models.

Chapter 1 starts out with the notions of systems, models, and discrete-event simulation. As its name suggests, discrete-event simulation views both time and state as discrete (rather than continuous) in the sense that its state trajectories are step functions. Thus, the prevailing state in a discrete-event simulation is always considered to be constant over some interval, and state transitions take place at discrete points in time from one discrete state to another. Consequently, the notions of *state transitions* (or *state jumps*) and *holding time* (or *sojourn time*) in a state are fundamental to discrete-event simulation. Philosophically, a discrete-event simulation simplifies reality by assuming that something "fundamentally new" occurs only at state transitions, whereas nothing "fundamentally new" or "unforeseen" takes place in between transitions. State transitions in a discrete-event simulation are triggered by *events*. In practice, events are fragments of computer code associated with an execution time; these are kept in chronological order in an *event list*, maintained by the simulation program. A *simulation clock* variable provides the notion of time. From a high-level viewpoint, a discrete-event simulation consists of a single repeatedly executed loop that first dequeues the current most imminent event from the event list, increments the simulation clock to the associated event time, and finally executes that event (in the course of which new events may be added to the event list).

Typically, a discrete-event simulation incorporates some randomness. From a high-level viewpoint, this randomness manifests itself in just two aspects: Either state transitions are random or the holding times are random or, most commonly, both are random. If a holding time is zero, then a succession of state transitions takes place instantaneously; the new simulation state will be well-defined, provided the number of instantaneous transitions is finite. At the applied level, the modeler generates random quantities (e.g., interarrival times and service times of customers in a queue) from prescribed distributions or probability laws. Chapter 2 covers computer generation of random variables and stochastic processes.

A large part of discrete-event simulation consists of so-called *output analysis*, namely, the collection of statistics from simulation runs, the formation of statistical estimates and confidence intervals for quantities of interest, and the performance of statistical tests in order to make decisions, based on statistical inference. Statistics collection and estimator formation are normally handled as part of event execution, while statistical inference is normally deferred until the end of the simulation or even relegated to separate software outside it. Chapter 3 covers basic output analysis techniques, including the *regenerative method* and *batch-means method.*

The smaller the variance of the statistical (output) estimators, the better (a smaller variance gives rise to narrower confidence intervals for a fixed confidence level). Conversely, if the confidence intervals are unsatisfactorily wide, the analyst

has two choices. The first choice is to reduce the variance by collecting additional observations. Naturally, this requires more or longer simulation runs with an attendant computational cost, typically in terms of increased computing time. An alternative approach is to change the experimental conditions and include more information about the system in such a way that the estimator is still valid, but the resulting variance is much smaller for a comparable computational effort. In that case, we say that *variance reduction* was attained. Chapter 4 explains basic variance reduction techniques.

CHAPTER 1

Systems, Models, and Simulation

1.1 INTRODUCTION

1.1.1 Discrete-Event Systems and Simulation

This book is about simulation and modeling of *discrete-event systems* (DES). All too often, real-life DES are too complex to model analytically, requiring the analyst to resort to statistical computer simulation models in order to approximate numerically their desired characteristics. Simulation has now become one of the most widespread modeling approaches in operations research, management science, and engineering. Furthermore, the sustained growth in size and complexity of emerging real-world systems (e.g., high-speed communications networks) will undoubtedly ensure that the popularity of computer simulation will continue to grow.

This chapter is organized as follows. Section 1.2 describes basic concepts, such as systems, models, simulation, and Monte Carlo methods. Finally, Section 1.3 deals with discrete-event simulation, and in particular, with one of its most important ingredients—the simulation clock.

1.2 SYSTEMS, MODELS, AND SIMULATION

This section describes the concepts of systems, models, and simulation. By a *system*, we mean a set of related entities, sometimes called *components* or *elements*. For instance, a hospital may be considered as a system, with doctors, nurses, and patients as elements. The elements possess certain characteristics or *attributes* that take on logical or numerical values. In our example, an attribute might be the number of beds, the number of X-ray machines, skill level, and so on. Typically, the activities of individual components interact in time. These activities cause changes in the system's state. For example, the state of a hospital's waiting room might be described by the number of patients waiting for a doctor. When a patient arrives at the hospital or leaves it, the system jumps to a new state.

We shall be solely concerned with discrete-event systems, to wit, those systems in which the state variables change instantaneously through jumps at discrete points in time; DES are qualitatively distinct from *continuous systems* (CS), where state variables change continuously with respect to time. Examples of a discrete and a continuous sytem are, respectively, a bank serving customers and a car moving on a freeway. In the former case, the number of waiting customers is a piecewise-constant state variable that changes only when either a new customer arrives at the bank or a customer finishes to transact his business and departs from the bank; in the latter case, the car velocity is a state variable that can change continuously in time.

The first step in studying a system is to build a model from which we can obtain predictions concerning the behavior of the system under study. The importance of models and model building has been discussed frequently. An apt quotation appears in Rosenbluth and Wiener (1945) who wrote:

> No substantial part of the universe is so simple that it can be grasped and controlled without abstraction. Abstraction consists in replacing the part of the universe under consideration by a model of similar but simpler structure. Models ... are thus a central necessity of scientific procedure.

By a *model*, we mean an abstraction of some real system that can be used to obtain predictions and formulate control strategies. In particular, models are used to analyze one or more changes in various aspects of the modeled system that may affect other aspects of the same system.

In order to be useful, a model must necessarily incorporate elements of two conflicting attributes—realism and simplicity. On the one hand, the model should serve as a reasonably close approximation to the real system and incorporate most of the important aspects of the real system. On the other hand, the model must not be overly complex so as to preclude its understanding and manipulation. Being a formalism, a model is necessarily an abstraction.

One may think that the more detailed the model is, the better it captures reality. However, adding excessive detail to a model tends to make the solution more difficult; at best it tends to convert the method for solving the problem at hand from an analytical to an approximate numerical one. Often, it is not even necessary for the model to capture all system characteristics. All we require is that there be a high *correlation* between model predictions and real-life system performance. To ascertain whether this requirement is satisfied or not, it is important to test the model's validity.

Usually, we begin testing a model by reexamining the formulation of the problem and uncovering possible flaws. Another check on the validity of a model is to ascertain that all mathematical expressions are dimensionally consistent. A third useful test consists of varying input parameters and checking that the output from the model behaves in a plausible manner. The fourth test is the so-called *retrospective* test. It involves using historical data to reconstruct the past and then determining how well the resulting solution would have performed, if it had been

used. Comparing the effectiveness of this hypothetical performance with what actually happened then indicates how well the model predicts "reality." However, a disadvantage of retrospective testing is that it uses the same data as the model. Unless the past is a representative replica of the future, it is better not to resort to this test at all. A crucial step in building an optimization-oriented model is the construction of the objective function, formulated as a mathematical function of the decision variables.

Having constructed some model for the problem under consideration, the next step is to derive a solution from this model. To this end, both *analytical* and *numerical* solution methods may be invoked. An analytical solution is usually obtained directly from its mathematical representation in the form of formulas. A numerical solution is generally an approximation via a suitable approximation procedure.

This book deals with numerical solution/estimation methods obtained via computer simulation. More precisely, we use stochastic simulation, which includes some randomness in the underlying model, rather than deterministic simulation (see below).

Computer simulation has long served as an important tool in a wide variety of disciplines, such as supersonic jet flight, telephone communications systems, wind tunnel testing, large-scale battle management (e.g., to evaluate defensive or offensive weapon systems), or maintenance operations (e.g., to determine the optimal size of repair crews), to mention a few. Although simulation is still sometimes viewed as a "method of last resort" to be employed when "everything else has failed," recent advances in simulation methodologies, software availability, sensitivity analysis, and stochastic optimization have combined to make simulation one of the most widely accepted and practiced tools in system analysis and operations research.

Naylor et al. (1966) define simulation as follows:

> Simulation is a numerical technique for conducting experiments on a digital computer, which involves certain types of mathematical and logical models that describe the behavior of business or economic system (or some component thereof) over extended periods of real time.

This definition is extremely broad, encompassing such diverse and seemingly unrelated enterprises as economic and financial modeling, wind tunnel testing of aircraft, war games, and business management games.

The motivation for simulation modeling is nicely put by Naylor et al. (1966):

> The fundamental rationale for using simulation is man's unceasing quest for knowledge about the future. This search for knowledge and the desire to predict the future are as old as the history of mankind. But prior to the seventeenth century the pursuit of predictive power was limited almost entirely to the purely deductive methods of such philosophers as Plato, Aristotle, Euclid, and others.

The following list of typical situations should give the reader some idea on where simulation would be an appropriate tool.

- The system may be so complex that a formulation in terms of simple mathematical equations may be impossible. Most economic systems fall into this category. For example, it is often virtually impossible to describe the operation of a business firm, an industry, or an economy in terms of a few simple equations. Another class of problems that leads to similar difficulties is that of large-scale complex queueing systems. Simulation has been an extremely effective tool for dealing with problems of this type.
- Even if a mathematical model can be formulated, which captures the behavior of some system of interest, it may not be possible to obtain a solution to the model by straightforward analytical techniques. Again, economic systems and complex queueing systems exemplify this type of difficulty.
- Simulation may be used as a pedagogical device for teaching both students and practitioners basic skills in system analysis, statistical analysis, and decision making. Among the disciplines in which simulation has been used successfully for this purpose are business administration, economics, medicine, and law.
- The formal exercise of designing a computer simulation model may be more valuable than the actual simulation itself. The knowledge obtained in designing a simulation study serves to crystallize the analyst's thinking and often suggests changes in the system being simulated. The effects of these changes can then be tested via simulation before implementing them in the actual system.
- Simulation can yield valuable insight into the problem of identifying which variables are important and which have negligible effect on the system, and can shed light on how these variables interact (see Chapters 5 and 6).
- Simulation can be used to experiment with new scenarios, so as to glean insight into system behavior under new circumstances.
- Simulation provides an *in-vitro* lab, allowing the analyst to discover better controls of the system under study.
- Simulation makes it possible to study dynamic systems in either real, compressed, or expanded time horizons.

As a modeling methodology, simulation modeling is by no means ideal. Some of its shortcomings and various caveats are:

- Simulation provides *statistical estimates* rather than the *exact* characteristics and performance measures of the model. Thus, simulation results are subject to uncertainty and contain "experimental errors."
- Simulation modeling is typically time-consuming and consequently expensive in terms of analyst time.

- Simulation results, no matter how precise, accurate, and impressive, provide consistently useful information about the actual system, *only* if the model is a "valid" representation of the system under study.

Computer simulation models can be classified in several ways:

1. *Static Versus Dynamic Models.* Static models are those that do not evolve in time, and therefore do not represent the passage of time. In contrast, dynamic models represent systems that evolve over time (e.g., traffic light operation).
2. *Deterministic Versus Stochastic Models.* If a simulation contains *only* deterministic (i.e., nonrandom) components, it is called *deterministic.* In a deterministic model, all mathematical and logical relationships between the elements (variables) are fixed in advance and not subject to uncertainty. A typical example is a complicated and analytically unsolvable system of standard differential equations, describing, say, a chemical reaction. In contrast, a model with at least one random input variable is called a *stochastic* model. Most queueing and inventory systems are modeled stochastically.
3. *Continuous Versus Discrete Simulation Models. Discrete* and *continuous* models are defined in the same way as discrete and continuous systems. Note that decisions on whether to use a discrete or continuous model for a given system depends on the objectives of the study. An example of a continuous simulation is a mathematical model aiming to calculate a numerical solution for a system of differential equations. Queueing network simulations are predominantly discrete simulations.

This book treats, in the main, discrete, dynamic, and stochastic simulation models, collectively called *discrete-event dynamic models* (DEDM), with the exception of Chapter 5 which addresses *discrete-event static models* (DESM). A DESM typically deals with evaluating (estimating) complex multidimensional integrals via the Monte Carlo method.

The term "Monte Carlo" was used by von Neumann and Ulam during World War II as a code word for the secret work at Los Alamos on problems related to the atomic bomb. That work involved simulation of random neutron diffusion in nuclear materials.

One of the earliest problems connected with the Monte Carlo method is Buffon's needle problem. This famous problem can be stated as follows. A needle of length l units is thrown randomly onto a floor composed of parallel planks of equal width of d units, where $d > l$. What is the probability that the needle, once it comes to rest, will cross (or touch) a crack separating the planks on the floor? It can be shown that the probability of the needle hitting a crack is $P = 2l/\pi d$, which can be estimated as the ratio of the number of throws hitting a crack to the total number of throws.

At the turn of the century, the Monte Carlo method was used to examine the Boltzmann equation. In 1908, one of the famous statistician's students used the Monte Carlo method for estimating the correlation coefficient in his t distribution.

For more details on the Monte Carlo method, see Ermakov (1976), Kalos and Wittlock (1986), and Rubinstein (1981).

It should be pointed out that recent advances in computer technology have enabled the use of *parallel* or *distributed* simulation, that is, the execution of discrete-event simulation on multiple linked (networked) computers operating simultaneously in a cooperative manner. Such an environment allows simultaneous "distribution" of different computing tasks among the individual processors, thus reducing the overall simulation time. For more details on this interesting and fast-developing area, see Heidelberger (1988) and Misra (1986) and references therein.

1.3 DISCRETE-EVENT SIMULATION

Discrete-event simulation deals with discrete-event dynamic systems (DEDS) modeling, and computes the evolution of their trajectories over time. Over an infinite time horizon, these systems change their state only at a countable number of time points. (In practical simulations, the number of state changes is, of course, finite). State changes are triggered by the execution of simulation events occurring at the corresponding time points; an *event* is a collection of attributes (values, flags, etc.), chief among which is the *event occurrence time* and an associated algorithm to execute state changes.

Example 1.3.1. Consider a single-server queue, say, an information desk in a hotel. Assume that we are interested in estimating the average number of customers waiting for information. The state variables for such a DES model might be the time of arrival of each customer and the status of the server (busy or idle). The arrival time of a customer is needed to compute the number of customers in the queue, which is incremented by one upon each customer arrival. The status of the server is needed to determine whether the newly arrived customer can be served immediately or must join the end of the queue. It is readily seen that there are two types of events in this simple system: customer arrival at the system and customer departure from it, immediately following service completion. Clearly, both are events, since they cause the number of customers to be incremented and decremented by one, respectively.

1.3.1 Simulation Clock and Event List

Because of their dynamic nature, DEDS require a time-keeping mechanism to *advance the simulated time* from one event to another, as the simulation unfolds in time. The variable recording the current simulation time is called the *simulation clock.* To keep track of events, the simulation maintains a list of all pending events. This list is called the *event list,* and its task is to maintain all pending events in chronological order, that is, events are inserted into it ordered by their time of occurrence. In particular, the most imminent event is always located at the head of the event list.

Initially, the simulation clock is set to zero, and the initial event(s) are loaded into the event list (chronologically ordered). Next, the most imminent event is unloaded from the event list for execution, and the simulation clock is advanced to its occurrence time. In the course of executing the current event, the state of the system is updated, and future events are typically generated and loaded into the event list. The process of unloading events from the event list, advancing the simulation clock, and executing the next most imminent event terminates when some specified stopping condition is met, say, as soon as a prescribed number of customers depart from the system.

This approach for advancing the simulation time is called the *next-event time advance* approach. The following example illustrates this approach for the single-server queueing model of Example 1.3.1.

Example 1.3.2. Define the following random variables (variates):

t_K is the clock time of arrival of customer K (by convention, $K_0 = 0$ is not a customer arrival time).

$A_K = t_K - t_{K-1}$ is the interarrival time, separating the arrivals of customer $(K-1)$ and customer K.

S_K is the service time of customer K; during this time, the server is unavailable to serve other customers.

D_K is the delay (sojourn time) of customer K in the system. (D_K equals the delay of customer K in the queue plus the service time S_K.)

$t'_K = t_K + D_K$ is the completion time, that is, the simulation clock time when customer K completed service and departed from the system.

E_K is the time of occurrence of the Kth event (by convention $E_0 = 0$ does not correspond to an event time).

V_t is the virtual time in the system (time to serve all customers) at time t.

Assume that the interarrival times, $A_1, A_2, \ldots$, and the service times, $S_1, S_2, \ldots$, are random variables, generated from some given cumulative distribution functions (cdf's) F_1 and F_2, respectively. Assume further that at time $t_0 = E_0 = 0$ the server is idle, so that at time t_1 the first customer arrives for service at an empty system (see Figure 1.1). The event list is initialized by an arrival event with occurrence time $t_1 = A_1$, and t_1 is determined by generating a random variable A_1 from F_1. This event is unloaded for execution, and the simulation clock is advanced from 0 to the time of the first event $E_1 = t_1$. Since the first customer arrives at an idle system, his delay in the system is $D_1 = S_1$, and the status of the server at time t_1 changes from idle to busy. Clearly, a service completion event will be scheduled at time $t'_1 = D_1$ and the next arrival event will take place at time $t_2 = t_1 + A_2$. If $t_2 < t'_1$, as depicted in Figure 1.1, the simulation clock is advanced from time E_1 to the next arrival event, that is, we set $E_2 = t_2$. Note that the number of customers in the system would be then incremented from 1 to 2. Since the customer arriving at time t_2 finds the server busy, we record t_2 and compute $t_3 = t_2 + A_3$, the time of the third arrival.

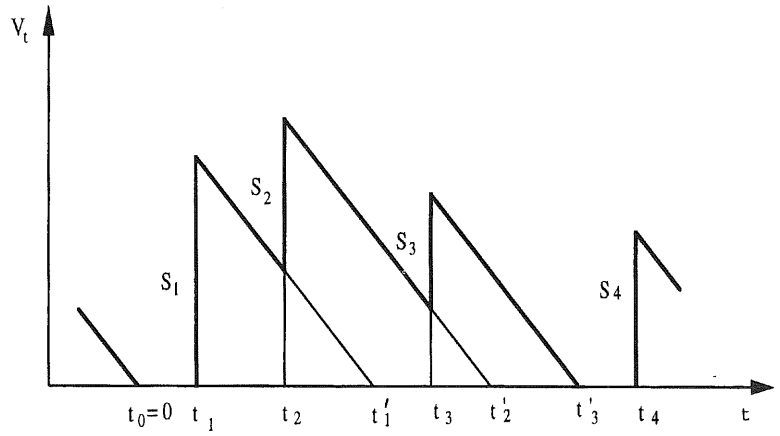

Figure 1.1. Sample path realization of the virtual time process, $\{V_t\}$, in a queueing system.

If $t_3 > t_1'$, as depicted in Figure 1.1, we advance the simulation clock from E_2 to the time of the next event, $E_3 = t_1'$. Executing this event means that the second customer completes his service and departs from the system, while the first customer in the queue (the second customer to arrive) begins service at the same time. At this point, we generate S_2 from F_2, calculate the sojourn time D_2 as $D_2 = t_1' - t_2 + S_2$ and decrement the number of customers in the system from 2 to 1. If $t_3 < t_2'$, as in Figure 1.1, the simulation clock is advanced from E_3 to the time of the next event, $E_4 = t_3$, and so forth. In Figure 1.1, we terminate our "simulation" after executing the event at time $E_7 = t_4$, which takes place when the fifth customer arrives at an idle server. The event time $E_7 = t_4$ follows event times $E_5 = t_2'$ and $E_6 = t_3'$.

Note that the event time $E_6 = t_3'$ corresponds to the time the server becomes idle again, having completed a busy period of length $t_3' - t_1$. The time between the onset of two consecutive busy periods is called a *cycle*. Clearly, each cycle is composed of a busy period and an idle period in succession. In our case (see Figure 1.1):

$$\text{cycle} = \text{busy period} + \text{idle period} = (t_4 - t_1) = (t_3' - t_1) + (t_4 - t_3').$$

Figure 1.1 actually depicts a sample path of the *virtual waiting time* (unfinished work), with V_t defined as the time to complete service for all customers present in the queueing system at time t. The evolution of the process $\{V_t\}$ is characterized by upward jumps (customer arrivals) followed by downward linear decreases (work depletion). Figure 1.2 depicts the associated sample path of the number of customers, N_t, present in the queueing system at time t. Sample paths of the process $\{N_t\}$ are piecewise constant with upward and downward unit jumps representing customer arrivals and departures, respectively.

More details on discrete-event simulation are found in Bratley et al. (1987), Banks and Garson (1984), Kleijnen (1987, 1991), Kleijnen and Van Groenendaal (1992), Law and Kelton (1991), Pegden (1989), Pollatschek (1996), Pritsker (1992), and Yakovitz (1977).

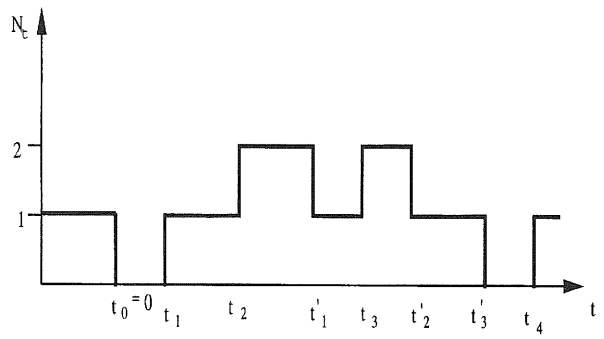

Figure 1.2. Sample path realization of the number of customers process, $\{N_t\}$, in a queueing system.

REFERENCES

Banks, J. and Carson, J. S. (1984). *Discrete-Event System Simulation*, Prentice-Hall, Englewood Cliffs, NJ.

Bratley, P., Fox, B. L., and Schrage, L. E. (1987). *A Guide to Simulation*, 2nd ed., Springer-Verlag, New York.

Ermakov, J. M. (1976). *Monte Carlo Method and Related Questions*, Nauka, Moscow (in Russian).

Feller, W. (1958). *An Introduction to Probability Theory and Its Application*, Vol. 1, Wiley, New York.

Heidelberger, P. (1988). "Discrete event simulations and parallel processing: Statistical properties," *SIAM J. Statist. Comput.*, **9**, 1114–1132.

Kleijnen, J. P. C. (1987). *Statistical Tools for Simulation Practitioners*, Marcel Dekker, New York.

Kleijnen, J. P. C. (1991). "Sensitivity analysis of simulation experiments: Tutorial on regression analysis and statistical design," Tilburg University, Tilburg, The Netherlands.

Kleijnen, J. P. C. and Van Groenendaal, W. (1992). *Simulation: A Statistical Perspective*, Wiley, Chichester.

Law, A. M. and Kelton, W. D. (1991). *Simulation Modeling and Analysis*, McGraw-Hill, New York.

Misra, J. (1986). "Distributed discrete-event simulation," *Computing Surveys*, **18**, 39–65.

Naylor, T. J., Balintfy, J. L., Burdick, D. S., and Chu, K. (1966). *Computer Simulation Techniques*, Wiley, New York.

Pegden, C. D. (1989). *Introduction to SIMAN* (January 1989 version), Systems Modeling Corp., Sewickley, PA.

Pollatschek, M. A. (1996). *Programming Discrete Simulations* R&D Books, Lawrence, KS.

Pritsker, A. A. B. (1992). *Introduction to Simulation and SLAM II*, 3rd ed., Systems Publishing, West Lafayette, IN.

Rosenbluth, A. and Wiener, N. (1945). "The role of models in science," *Phil. Sci.*, **XII**(4), 316–321.

Rubinstein, R. Y. (1981). *Simulation and the Monte Carlo Method*, Wiley, New York.

Yakowitz, S. J. (1977). *Computational Probability and Simulation*, Addison-Wesley, Reading, MA.

CHAPTER 2

Random Numbers, Variates, and Stochastic Process Generation

This chapter deals with computer generation of pseudorandom numbers (random numbers, for short) and random variables (variates, for short). In this book, the terms random variables and variates are largely interchangeable; however, the former term connotes the formal concept, while the latter is used mainly in simulation context as the computer-generated realization of the former. Section 2.1 presents a brief discussion on generation of variate sequences, whose marginals are uniformly distributed on the interval (0, 1); such random numbers are available on most computers. Section 2.2 deals with generation methods of one-dimensional variates from prescribed distributions: the *inverse-transform* method, the *composition* method, and the *acceptance-rejection* method. Section 2.3 deals with the multidimensional case. In particular, Section 2.3.1 discusses cases where the one-dimensional inverse-transform method can be extended to a multidimensional version. Section 2.3.2 deals with the multidimensional version of the acceptance-rejection method for generating random vectors from arbitrary distributions, while Section 2.3.3 describes random vector generation from multinormal distributions. Sections 2.4 and 2.5 describe specific generation methods for continuous and discrete variates, respectively, while Section 2.6 deals with generation of stochastic processes. Section 2.7 describes generation of points, uniformly distributed over different regions. Finally, Section 2.8 deals with generation of autocorrelated sequences; it describes a versatile class of stochastic sequences, called TES processes, and input analysis based on them.

2.1 RANDOM NUMBER GENERATION

In a typical stochastic simulation, randomness is introduced into simulation models via sequences $U_1, U_2, \ldots,$ of independent random variables, uniformly distributed on the interval (0, 1). This chapter details how such random numbers are generated and discusses some of the generator properties. These random numbers are used to

generate samples of random variables from specified distributions (Devroye, 1986) in order to characterize system behavior. For example, for a queueing model, one needs to specify the interarrival and service time distributions in order to drive the simulation over time; and for a stochastic inventory model, one needs to specify the distribution of random demands.

In the early days of simulation, randomness was generated by *manual* techniques, such as coin flipping, dice rolling, card shuffling, and roulette spinning. Later on, *physical devices*, such as noise diodes and Geiger counters, were attached to computers for the same purpose. The prevailing belief held that *only* mechanical or electronic devices could produce "truly" random sequences. Although mechanical devices are still widely used in gambling and lotteries, these methods were abandoned by the computer simulation community for several reasons: Mechanical methods are too slow for general use; the generated sequences cannot be reproduced; and, most importantly, it has been found that the generated numbers exhibit both bias and dependence.

As computer simulation is becoming ever more widespread, increasing attention has been paid to random number generators (RNG), which are suitable for computer implementation. Perhaps the earliest such method is due to von Neumann who suggested that an n-digit number be squared and that the middle n digits of the result be used as the next number in the sequence. Most RNGs are quite fast, can readily reproduce a given sequence of random numbers, and require little storage space. In fact, the RNGs used in practice produce *deterministic* sequences. Still, they capture certain statistical properties, mimicking *true* random sequences. For this reason, the RNGs are called *pseudorandom*. In practice, however, it is sufficient that the joint distributions of a *pseudorandom sequence* $(U_{i+1}, \ldots, U_{i+k})$, $i = 1, 2, \ldots$, be close to uniformity over $(0, 1)^k$, say, for $k \leq 6$ (see Ripley, 1983).

The most common methods for generating pseudorandom sequences use the so-called *linear congruential generators*, introduced in Lehmer (1951). These generate a deterministic sequence of numbers by means of a recursive formula that is based on calculating the modulo-m residues of a linear transformation, for some integer m. More precisely, congruential generators are based on a fundamental congruence relationship, given by

$$X_{i+1} = aX_i + c \pmod{m}, \tag{2.1.1}$$

where the initial value, X_0, is called the *seed*, a and m are positive integers, and c is a nonnegative integer.

Note that applying the modulo-m operator in (2.1.1) means that $aX_i + c$ is divided by m, and the remainder is taken as the value of X_{i+1}. That is, each X_i can only assume a value from the set $\{0, 1, \ldots, m-1\}$, and the quantities

$$U_i = \frac{X_i}{m}, \tag{2.1.2}$$

called *pseudorandom numbers*, constitute approximations to the true sequence of uniform random variables. In the special case where $c = 0$, formula (2.1.1) simply reduces to

$$X_i = aX_i \pmod{m}. \tag{2.1.3}$$

Such a generator is called a *multiplicative congruential generator* to distinguish it from (2.1.1), which is called a *mixed congruential generator* (as it involves both a multiplicative and an additive term).

A recursive scheme of the form (2.1.1) or (2.1.3) is called a *congruential method*; the integers a, c, and m are called the *multiplier*, the *increment*, and the *modulus*, respectively. Sequences thus generated are called pseudorandom sequences.

It follows from (2.1.1) and (2.1.2) that the sequence $\{U_i\}$ is completely deterministic, provided the four values X_0, a, c, and m are fixed. Although the sequence $\{U_i\}$ is completely deterministic, we may accept it as a random sequence if it appears to be "sufficiently random" in the sense that the generated numbers, $U_1, U_2, \ldots$, appear to be uniformly distributed and statistically independent. It is readily seen that an arbitrary choice of X_0, a, c and m will not lead to a pseudorandom sequence with good statistical properties. In fact, number theory has been used to show that only a few combinations of these produce satisfactory results.

Note also that the sequence $\{X_i\}$ will repeat itself in at most m steps and will therefore be periodic with period not exceeding m. For example, let $a = c = X_0 = 3$ and $m = 5$; then the sequence obtained from the recursive formula $X_{i+1} = 3X_i + 3 \pmod 5$ is $X_i = 3, 2, 4, 0, 3$. Indeed, the period of the generator in the above example is 4, while $m = 5$.

Because of the deterministic character of the sequence, the entire sequence recurs as soon as any number is repeated. This is referred to as "getting into a loop," that is, there is a cycle of numbers that is repeated endlessly. Knuth (1981) shows that all sequences of the form $X_{i+1} = f(X_i)$ get into a loop. One wants, of course, to choose m as large as possible to ensure a sufficiently long sequence of distinct numbers in a cycle. To do so, let p be the period of the sequence. (When p equals its maximum m, we say that the random number generator has a *full period*.) It is shown in Knuth (1981) that the generator defined in (2.1.1) has a full period, m, if and only if:

1. c is *relatively prime* to m, that is, c and m have no common divisors greater than one.
2. $a \equiv 1 \pmod g$ for every prime factor g of m.
3. $a \equiv 1 \pmod 4$ if m is a multiple of 4.

(Readers not acquainted with the congruence relation $\equiv$, should read $=$ instead.) Condition 1 means that the greatest common divisor of c and m is unity. Condition 2 means that $a = g\lfloor a/g \rfloor + 1$. Let g be a prime factor of m; then letting

$k = \lfloor a/g \rfloor$, one has

$$a = 1 + gk. \tag{2.1.4}$$

Condition 3 means that

$$a = 1 + 4\lfloor a/4 \rfloor \tag{2.1.5}$$

if $m/4$ is an integer.

In computer implementations, m is selected as a large prime number that can be accommodated by the computer word size. For example, in a binary 32-bit word computer, statistically acceptable generators can be obtained by choosing $m = 2^{31} - 1$ and $a = 7^5$, provided the first bit is a sign bit. A 64-bit word computer or even a 48-bit word computer will naturally yield better statistical results.

Formulas (2.1.1), (2.1.2), and (2.1.3) can be readily extended to pseudorandom vector generation. For example, the multidimensional version of (2.1.3) and (2.1.2) can be written as

$$\boldsymbol{X}_{i+1} = \boldsymbol{A}\boldsymbol{X}_i \pmod{\boldsymbol{M}}, \tag{2.1.6}$$

and

$$\boldsymbol{U}_{i+1} = \boldsymbol{M}^{-1}\boldsymbol{X}_i, \tag{2.1.7}$$

respectively, where $\boldsymbol{A}$ is a nonsingular $n \times n$ matrix, $\boldsymbol{M}$ is a vector of constants of the same dimensionality as the vector $\boldsymbol{X}$, and $\boldsymbol{M}^{-1}\boldsymbol{X}_i$ is an n-dimensional vector with components $M_1^{-1}X_1, \ldots, M_n^{-1}X_n$. The properties of pseudorandom vector generators and the choice of their parameters, $\boldsymbol{M}$ and $\boldsymbol{A}$, are discussed in Section 10.1 of Niederreiter (1992).

Beside the linear congruential generators, other classes have been proposed, so as to achieve longer periods and better statistical properties (see L'Ecuyer, 1990).

A number of statistical tests of randomness for pseudorandom sequences, such as the serial test, run test, spectral test, chi-squared goodness-of-fit test, and the Kolmogorov-Smirnov test, have been proposed and discussed in the literature on random number generation (e.g., Law and Kelton, 1991).

Most computer languages already contain a built-in pseudorandom number generator. The user is typically requested only to input the initial seed, X_0, and upon invocation, the RNG produces a sequence of independent uniform (0, 1) variates. Instead of exploring the theoretical and practical aspects of pseudorandom number generation [see Law and Kelton (1991), L'Ecuyer and Tezuka (1991), L'Ecuyer (1990), Niederreiter (1991, 1992), Knuth (1981) and references therein], we assume the availability of a "black box," capable of producing pseudorandom numbers on demand.

2.2 VARIATE GENERATION FROM PRESCRIBED DISTRIBUTIONS

This section treats generation of one-dimensional variates from a prescribed distribution. We consider here the inverse-transform method, the composition method, and the acceptance-rejection method.

In the algorithms outlined below, the keyword "return" signals that the requisite random number has been obtained. The algorithm terminates implicitly if only a single observation is desired; otherwise, it is understood to loop back to the beginning to compute more random numbers. The notation $X \sim F$ means that the variate X has distribution F.

2.2.1 Inverse-Transform Method

Let X be a variate with cumulative distribution function (cdf) $F(x)$. Since $F(x)$ is a nondecreasing function, the inverse function $F^{-1}(y)$ may be defined as

$$F^{-1}(y) = \inf\{x: F(x) \geq y\}, \qquad 0 \leq y \leq 1. \tag{2.2.1}$$

(Readers not acquainted with the notion of inf (infimum) should read min).

It is easy to prove that if U is uniformly distributed over the interval [0, 1] [denoted, hereafter, by $\mathcal{U}(0, 1)$], then (Figure 2.1)

$$X = F^{-1}(U) \tag{2.2.2}$$

has cdf $F(x)$.

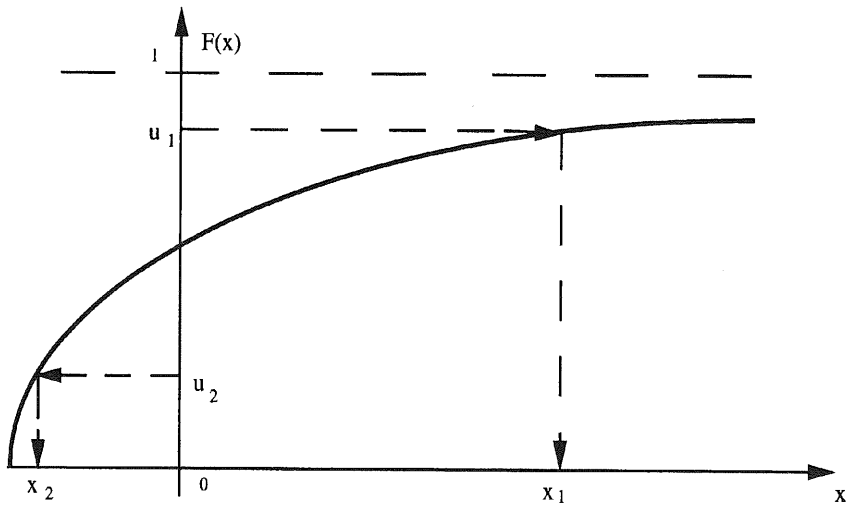

Figure 2.1. Inverse-transform method.

In view of the fact that F is invertible and $P(U \leq u) = u$, we readily obtain

$$P(X \leq x) = P[F^{-1}(U) \leq x] = P[U \leq F(x)] = F(x). \tag{2.2.3}$$

Thus, to generate a value, say x, of a random variate X with cdf $F(x)$, first sample a value, say u, of a uniform variate, U, compute $F^{-1}(u)$, and set it equal to x.

Figure 2.1 illustrates the inverse-transform method given by the following algorithm.

Algorithm 2.2.1 Inverse-Transform Method

1. *Generate U from $\mathcal{U}(0, 1)$.*
2. *Return $X = F^{-1}(U)$.*

Example 2.2.1. Generate a variate from the probability density function (pdf)

$$f(x) = \begin{cases} 2x, & 0 \leq x \leq 1 \\ 0, & \text{otherwise.} \end{cases} \tag{2.2.4}$$

The cdf is

$$F(x) = \begin{cases} 0, & x < 0 \\ \displaystyle\int_0^x 2x\,dx = x^2, & 0 \leq x \leq 1 \\ 1, & x > 1. \end{cases}$$

Applying (2.2.2), we have

$$X = F^{-1}(U) = U^{1/2}, \qquad 0 \leq u \leq 1.$$

Therefore, to generate a variate X from the pdf (2.2.4), first generate a variate U from $\mathcal{U}(0, 1)$, and then take its square root.

Example 2.2.2. Generate variates from the order statistics $Y_n = \max(X_1, \ldots, X_n)$ and $Y_1 = \min(X_1, \ldots, X_n)$, respectively, where $X_1, \ldots, X_n$ are independent and identically distributed (iid) variates with cdf $F(x)$.

The distributions of Y_n and Y_1 are

$$F_n(y) = [F(y)]^n$$

and

$$F_1(y) = 1 - [1 - F(y)]^n,$$

respectively. Applying (2.2.2), we get

$$Y_n = F^{-1}(U^{1/n})$$

and, since $1 - U$ is also from $\mathcal{U}(0, 1)$,

$$Y_1 = F^{-1}(1 - U^{1/n}).$$

In the special case where $X = U$ we have

$$Y_n = U^{1/n}$$

and

$$Y_1 = 1 - U^{1/n}.$$

Example 2.2.3 Generating Variates from a Discrete Distribution. Let X be a discrete variate with probability mass function (pmf)

$$p(x_i) = P(X = x_i), \qquad i = 1, 2, \ldots,$$

and cdf

$$F(x_k) = \sum_{x_i \leq x_k} p(x_i), \qquad k = 1, 2, \ldots,$$

respectively.

The algorithm for generating a variate from $F(x)$ (see Figure 2.2) can be written as:

Algorithm 2.2.2 Inverse-Transform Method for a Discrete Distribution

1. *Generate* $U \sim \mathcal{U}(0, 1)$.
2. *Find the smallest positive integer,* k, *such that* $U \leq F(x_k)$ *and return* $X = x_k$.

Much of the execution time in Algorithm 2.2.2 is spent in making the comparisons of step 2. This time can be reduced by using efficient search techniques (see Devroye, 1986).

Example 2.2.4 Generating Variates from an Empirical Distribution. Assume that our data can be grouped to fall into n adjacent intervals $[a_0, a_1)$,

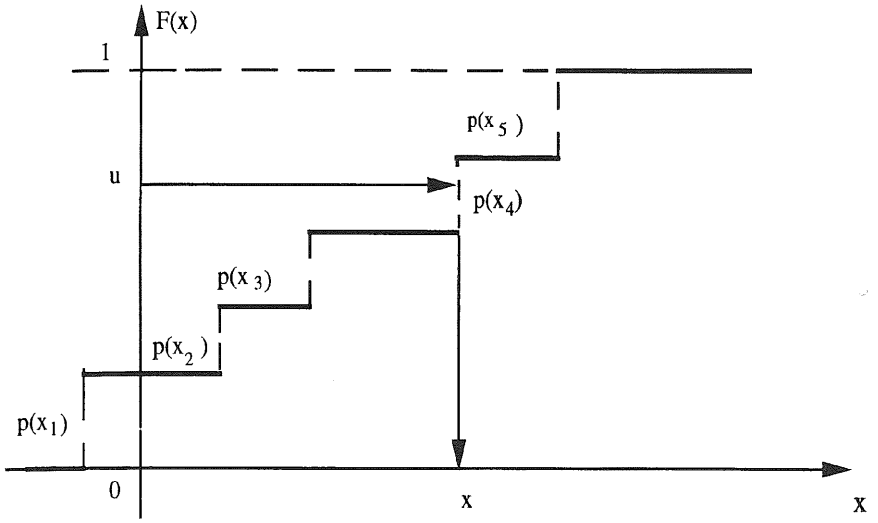

Figure 2.2. Inverse-transform method for a discrete variate.

$[a_1, a_2), \ldots, [a_{n-1}, a_n]$ and define the following piecewise-linear empirical distribution function:

$$F(x) = \begin{cases} 0, & x < a_0 \\ F(a_{i-1}) + \dfrac{x - a_{i-1}}{a_i - a_{i-1}}[F(a_i) - F(a_{i-1})], & a_{i-1} \le x < a_i \text{ for some } 1 \le i \le n \\ 1, & x > a_n, \end{cases}$$

where $F(a_0) = 0$, $F(a_i) = \sum_{j=1}^{i} r_j/r$, $i = 1, 2, \ldots, n$, r_i is the number of observations in the ith interval $[a_{i-1}, a_i)$ and $r = \sum_{i=1}^{n} r_i$.

The algorithm for generating a variate from an empirical distribution $F(x)$ (see Figure 2.3) can be written as:

Algorithm 2.2.3 Inverse-Transform Method for an Empirical Distribution (Histogram)

1. *Generate* $U \sim \mathcal{U}(0, 1)$.
2. *Find the smallest positive integer* k $(0 \le k \le n - 1)$, *such that* $U \le F(x_k)$, *and return*

$$X = F^{-1}(U) = a_k + [U - F(a_k)](a_{k+1} - a_k)/[F(a_{k+1}) - F(a_k)].$$

In general, the inverse-transform method requires that the underlying cdf, $F(x)$, exist in a form for which the corresponding inverse function $F^{-1}(\cdot)$ can be found analytically or algorithmically. Applicable distributions are, for example, exponential, uniform, Weibull, logistic, and Cauchy. Unfortunately, for many

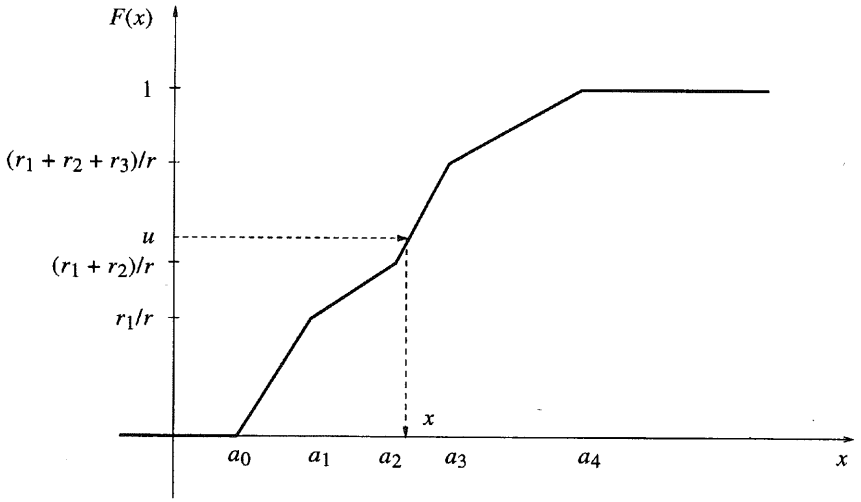

Figure 2.3. Inverse-transform method for an empirical distribution.

probability distributions, it is either impossible or difficult to find the inverse transform, that is, to solve

$$F(x) = \int_{-\infty}^{x} f(t)\,dt = u$$

with respect to x. Even in the case when F^{-1} exists in an explicit form, the inverse-transform method may not necessarily be the most efficient variate generation method (see Devroye, 1986).

*2.2.2 Composition Method

This method assumes that a cdf, $F(x)$, can be expressed as a probabilistic mixture (convex combination) of distributions, H_i, namely

$$F(x) = \sum_{i=1}^{n} p_i H_i(x), \tag{2.2.5}$$

where

$$0 < p_i \leq 1, \qquad \sum_{i=1}^{n} p_i = 1.$$

Let X_i have distribution function H_i, for $1 \leq i \leq n$. Then the random variable X with cdf $F(x)$ can be represented as

$$X = \begin{cases} X_1 \text{ with probability } p_1, \\ \quad\vdots \\ X_n \text{ with probability } p_n. \end{cases}$$

It is readily seen that in order to generate a variate X from $F(x)$, we must first generate a discrete variate Y, with discrete pdf $P(Y = i) = p_i$, $i = 1, \ldots, n$, and then generate the requisite variate, X; equivalently, given $Y = i$, X has distribution function $H(x|i) = P(X \leq x|Y = i)$. To see that, note that using conditioning, we can rewrite (2.2.5) as

$$F(x) = \sum_{i=1}^{n} P(X \leq x|Y = i)P(Y = i). \tag{2.2.5}$$

This approach is called the composition method. The corresponding algorithm is straightforward.

Algorithm 2.2.4 Composition Method

1. *Generate a variate (positive integer) Y from the discrete pdf:*

$$P(Y = i) = p_i, \qquad i = 1, \ldots, n.$$

2. *Return X from the cdf $H(x|i)$.*

The reader should show that X has a cdf given by 2.2.5.

The composition method can be readily extended to $n = \infty$ and to the case where Y is a continuous variate and the pdf of X can be expressed as

$$f(x) = \int h(x|y)p(y)\, dy.$$

Example 2.2.5. Generate a variate from

$$f(x) = n \int_1^{\infty} y^{-n} e^{-yx}\, dy.$$

In this case, we define the pdf's $h(x|y)$ and $p(y)$ as

$$h(x|y) = ye^{-yx}, \qquad x \geq 0$$

and

$$p(y) = \frac{n}{y^{n+1}}, \qquad 1 \leq y \leq \infty, \quad n \geq 1,$$

respectively. According to Algorithm 2.2.4, we first generate a variate Y from $p(y)$. Once this Y is determined, we next generate the desired variate X from the conditional pdf $h(x|y) = ye^{-yx}$, $x \geq 0$.

In summary, in order to generate a variate from a prescribed pdf, $f(x)$, we need to perform the following steps:

1. Generate independent variates U_1 and U_2 from $\mathcal{U}(0, 1)$.
2. Generate a variate Y from the pdf $p(y)$, that is, take $Y = F_Y^{-1}(U_1) = U_1^{-1/n}$. where F_Y is the cdf of Y.
3. Generate a variate X from the conditional cdf $H(x|y)$; equivalently, X is the solution of $U_2 = H^{-1}(X|Y)$ which equals $X = -(1/Y)\ln U_2$ [recall that if U is from $\mathcal{U}(0, 1)$, then so is $1 - U$].

2.2.3 Acceptance-Rejection Method

The inverse-transform and the composition methods are direct methods in the sense that they deal directly with the cdf of the variate to be generated. The acceptance-rejection method (ARM), to be presented next, is an indirect method, due to John von Neumann. It may be appealed to when the above-mentioned direct methods either fail or turn out to be computationally inefficient. An ARM, however, calls for specifying a function ϕ that *majorizes* the original pdf, $f(x)$.

To carry out the acceptance-rejection method, we represent $f(x)$ as

$$f(x) = Ch(x)g(x), \tag{2.2.6}$$

where $C \geq 1$, $h(x)$ is a pdf, $\phi(x) = Ch(x)$ majorizes the pdf $f(x)$ [i.e., $Ch(x) \geq f(x)$ for all x], and $0 < g(x) = f(x)/\phi(x) \leq 1$ (see Figure 2.4). Now, generate two variates, U from $\mathcal{U}(0, 1)$ and Y from $h(y)$, and test the inequality $U \leq g(Y)$. If the inequality holds, then we accept Y as the requisite variate from $f(x)$; otherwise, we reject the pair (U, Y) and try again until successful. The acceptance-rejection algorithm can be written as follows.

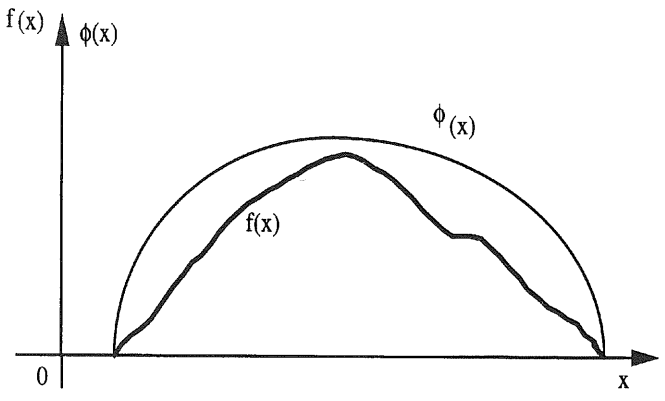

Figure 2.4. Acceptance-rejection method.

Algorithm 2.2.5 Acceptance-Rejection Method

1. *Generate U from $\mathcal{U}(0, 1)$.*
2. *Generate Y from $h(y)$, independent of U.*
3. *If $U \leq g(Y)$, return $X = Y$. Otherwise, go to step 1.*

Example 2.2.6 (Example 2.2.1 continued). We shall show how to generate a variate from the pdf

$$f(x) = \begin{cases} 2x, & 0 \leq x \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

using the acceptance-rejection method. For simplicity, take $h(y) = 1$, $(0 \leq y \leq 1)$ and $C = 2$. In this case, $g(x) = \frac{1}{2}f(x) = x$, and Algorithm 2.2.5 becomes:

1. Generate U from $\mathcal{U}(0, 1)$.
2. Generate Y from $\mathcal{U}(0, 1)$ independent on U.
3. If $U \leq Y$, return $X = Y$. Otherwise, go to step 1.

Note that this example is merely illustrative, since the inverse-transform method handles it efficiently.

The theoretical basis of ARM is provided by the following theorem:

Theorem 2.2.1. *Let X be a random variable distributed according to the pdf $f(x)$. Assume that $f(x)$ can be represented as*

$$f(x) = Ch(x)g(x),$$

where $C \geq 1$, $h(x)$ is a pdf, and $0 < g(x) \leq 1$. Let U and Y be distributed according to $\mathcal{U}(0, 1)$ and $h(y)$, respectively. Then

$$h(x|U \leq g(Y)) = f(x), \tag{2.2.7}$$

where $h(x|U \leq g(Y)) = (\partial/\partial x)P(Y \leq x|U \leq g(Y))$.

Proof. By Bayes's formula

$$h(x|U \leq g(Y)) = \frac{P(U \leq g(Y)|Y = x)h(x)}{P(U \leq g(Y))}. \tag{2.2.8}$$

Direct computations yield

$$P(U \leq g(Y)|Y = x) = P(U \leq g(x)) = g(x) \tag{2.2.9}$$

and

$$P(U \le g(Y)) = \int P(U \le g(Y)|Y = x)h(x)\,dx$$
$$= \int g(x)h(x)\,dx = \int \frac{f(x)}{C}\,dx = \frac{1}{C}. \tag{2.2.10}$$

Upon substitution of (2.2.9) and (2.2.10) into (2.2.8), we obtain

$$h(x|U \le g(Y)) = Ch(x)g(x) = f(x).$$

The efficiency of the acceptance-rejection method is determined by the probability $p = P(U \le g(Y))$ [see (2.2.10)]. Since the trials are independent, the probability of success in each trial is $p = 1/C$. The number of trials, N, before a successful pair (U, Y) occurs has a geometric distribution,

$$P(N = n) = p(1-p)^{n-1}, \qquad n = 1, \ldots, \tag{2.2.11}$$

with the expected number of trials equal to $1/p = C$.

For this method to be of practical interest, the following criteria must be used in selecting $h(x)$:

1. It should be easy to generate a variate from $h(x)$.
2. The efficiency, $1/C$, of the procedure should be large, that is, C should be close to 1 [which occurs when $h(x)$ is close to $f(x)$].

The maximal efficiency is attained when $f(x) = \phi(x)$, for all x. In this case, $1/C = C = 1$, $g(x) = 1$, $h(x) = f(x)$ and the acceptance-rejection method is obviated, since generating a variate from $f(x)$ is the same as generating it from $h(x)$.

Example 2.2.7. Generate a variate from

$$f(x) = \frac{2}{\pi R^2}\sqrt{R^2 - x^2}, \qquad -R \le x \le R.$$

Take $h(x) = 1/2R(-R < x < R)$ and $C = 4/\pi$. Then the inequality $f(x) \le 2/\pi R = Ch(x) = \phi(x)$ holds, $g(x) = f(x)/\phi(x) = \sqrt{1 - (x/R)^2}$, and Algorithm 2.2.5 becomes:

1. Generate two independent variates U_1 and U_2 from $\mathcal{U}(0, 1)$.

2. Use U_2 to generate Y from $h(y)$ via the inverse-transform method, namely, $Y = (2U_2 - 1)R$, and calculate

$$g(Y) = \frac{f(Y)}{\phi(Y)} = \sqrt{1 - (2U_2 - 1)^2}. \tag{2.2.10}$$

3. If $U_1 \le g(Y)$, which is equivalent to $(2U_2 - 1)^2 \le 1 - U_1^2$, then return $X = Y = (2U_2 - 1)R$; otherwise go to step 1.

It can be shown that the expected number of trials is $C = 4/\pi$, and the efficiency is $1/C = \pi/4 = 0.785$.

2.3 RANDOM VECTOR GENERATION

2.3.1 Vector Inverse-Transform Method

Suppose that we wish to generate a random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$ from a given cdf $F(\boldsymbol{x})$. We distinguish between the following two cases: (1) the variates $X_1, \ldots, X_n$ are *independent*, and (2) the variates $X_1, \ldots, X_n$ are *dependent*.

1. For independent variates $X_1, \ldots, X_n$, the joint pdf, $f(\boldsymbol{x})$, is

$$f(\boldsymbol{x}) = f(x_1, \ldots, x_n) = \prod_{i=1}^{n} f_i(x_i), \tag{2.3.1}$$

where $f_i(x_i)$ is the marginal pdf of the variate X_i. It is easy to see that in order to generate the random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$ from cdf $F(\boldsymbol{x})$, we can apply the inverse-transform method

$$X_i = F^{-1}(U_i), \qquad i = 1, \ldots, n, \tag{2.3.2}$$

for each variate separately.

Example 2.3.1. Let X_i, $1 \le x_i \le n$, be independent variates with the pdf's

$$f_i(x_i) = \begin{cases} \dfrac{1}{b_i - a_i}, & a_i \le x_i \le b_i, \quad i = 1, \ldots, n \\ 0, & \text{otherwise.} \end{cases}$$

To generate the random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$ with the joint pdf

$$f(x_1, \ldots, x_n) = \begin{cases} \displaystyle\prod_{i=1}^{n} \frac{1}{(b_i - a_i)}, & (x_1, \ldots, x_n) \in D \\ 0, & \text{otherwise} \end{cases}$$

where $D = \{(x_1, \ldots, x_n)\colon a_i \le x_i \le b_i, \quad i = 1, \ldots, n\}$, we apply the inverse-transform formula (2.3.2) and get $X_i = a_i + (b_i - a_i)U_i$, $i = 1, \ldots, n$, where $U_1, \ldots, U_n$ are iid from $\mathcal{U}(0, 1)$.

2. For dependent variates $X_1, \ldots, X_n$, represent the joint pdf $f(x)$ as

$$f(x_1, \ldots, x_n) = f_1(x_1) f_2(x_2|x_1) \cdots f_n(x_n|x_1, \ldots, x_{n-1}), \tag{2.3.3}$$

where $f_1(x_1)$ is the marginal pdf of X_1, and $f_k(x_k|x_1, \ldots, x_{k-1})$ is the conditional pdf of X_k given $X_1 = x_1, X_2 = x_2, \ldots, X_{k-1} = x_{k-1}$. Let $U_1, \ldots, U_n$ be iid variates from $\mathcal{U}(0, 1)$. It is readily seen that the vector $\boldsymbol{X} = (X_1, \ldots, X_n)$, obtained from the solution of the following system of equations

$$\begin{aligned} F_1(X_1) &= U_1 \\ F_2(X_2|X_1) &= U_2 \\ &\vdots \\ F_n(X_n|X_1, \ldots, X_{n-1}) &= U_n. \end{aligned} \tag{2.3.4}$$

is distributed according to the joint cdf, $F(x)$.

The algorithm for generating random vectors from (2.3.3) consists of just two steps:

Algorithm 2.3.1 Vector Inverse-Transform Method

1. *Generate n iid variates,* $U_1, \ldots, U_n$, *from* $\mathcal{U}(0, 1)$.
2. *Solve the system of Eqs. (2.3.4) with respect to* $\boldsymbol{X} = (X_1, \ldots, X_n)$.

The applicability of this method is quite limited since it requires knowledge of both marginal cdf's and conditional cdf's. This knowledge is typically unavailable in complex discrete-event systems.

2.3.2 Vector Acceptance-Rejection Method

Algorithm 2.2.5 and Theorem 2.2.1 are directly applicable to the multidimensional case. We need only bear in mind that $\boldsymbol{Y}$ (see step 2 of algorithm 2.2.5) becomes an n-dimensional random vector rather than a (unidimensional) random variable. Consequently, we need a convenient way of generating $\boldsymbol{Y}$ from the multidimensional pdf $h(\boldsymbol{y})$, say, by using the vector inverse-transform method. The following examples demonstrate the vector version of the acceptance-rejection method.

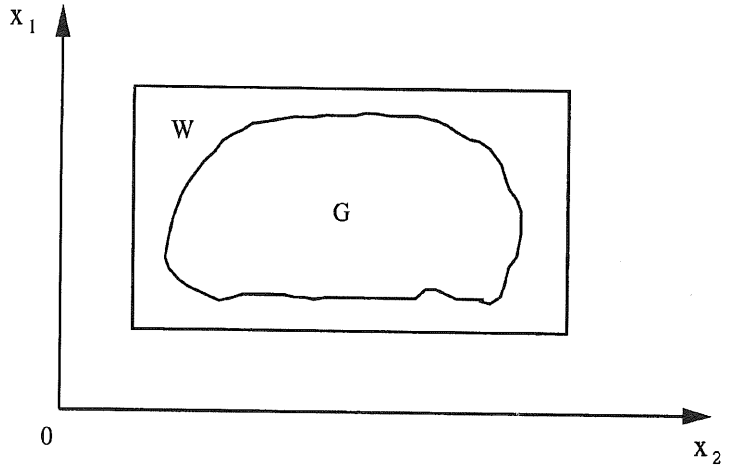

Figure 2.5. Vector acceptance-rejection method.

Example 2.3.2. Generate a random vector, uniformly distributed over the "irregular" two-dimensional region G (see Figure 2.5). The corresponding algorithm is straightforward.

1. Generate a random vector, $\boldsymbol{Y}$, uniformly distributed in W, where W is a "regular" region (multidimensional hypercube, hyper-rectangle, hyper-sphere, hyperellipsoid, etc.).
2. If $\boldsymbol{Y} \in G$, accept $\boldsymbol{Y}$ as a random vector, uniformly distributed over G; otherwise go to step 1.

As a special case, let G be the n-dimensional unit sphere, and let W be the n-dimensional hypercube, $\{-1 \le x_i \le 1\}_{i=1}^n$.

To generate a random vector, uniformly distributed over the surface of the n-dimensional unit sphere, we generate a random vector, uniformly distributed over the n-dimensional hypercube, $\{-1 \le x_i \le 1\}_{i=1}^n$, and then accept or reject the sample $(X_1, \ldots, X_n)$, depending on whether the point $(X_1, \ldots, X_n)$ is inside or outside the n-dimensional sphere. The corresponding algorithm is as follows:

Algorithm 2.3.2

1. *Generate* $U_1, \ldots, U_n$ *as iid variates from* $\mathcal{U}(0, 1)$.
2. *Set* $X_1 \leftarrow 1 - 2U_1, \ldots, X_n \leftarrow 1 - 2U_n$, *and* $Y^2 \leftarrow \sum_{i=1}^n X_i^2$.
3. *If* $Y^2 < 1$, *accept* $\boldsymbol{Z} = (Z_1, \ldots, Z_n)$, *where* $Z_i = X_i/Y$, $i = 1, \ldots, n$, *as the desired vector; otherwise go to step 1.*

Remark 2.3.1. To generate a random vector, uniformly distributed over the surface and interior of an n-dimensional unit sphere, we need only rewrite step 3 in

Algorithm 2.3.2 as:

3′. *If* $Y^2 \leq 1$, *accept* $Y = (Y_1, \ldots, Y_n)$ *as the desired vector.*

The efficiency of the vector acceptance-rejection method is equal to the ratio

$$\frac{1}{C} = \frac{\text{volume of the sphere}}{\text{volume of the hypercube}} = \frac{1}{n2^{n-1}} \frac{\pi^{n/2}}{\Gamma(n/2)},$$

where the volume of the unit sphere and hypercube are $\pi^{n/2}/[(n/2)\Gamma(n/2)]$ and 2^n, respectively. For even n ($n = 2m$)

$$\frac{1}{C} = \frac{\pi^m}{m!2^{2m}} = \frac{1}{m!}\left(\frac{\pi}{2}\right)^m 2^{-m} \to 0 \text{ as } m \to \infty.$$

In other words, the acceptance-rejection method grows inefficient in n and is asymptotically useless.

*2.3.3 Generating Variates from the Multinormal Distribution

A random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$ has a multinormal distribution, denoted by $N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, if its pdf is given by

$$f(x) = \frac{1}{(2\pi)^{n/2}|\boldsymbol{\Sigma}|^{1/2}} \exp[-\tfrac{1}{2}(x-\mu)'\boldsymbol{\Sigma}^{-1}(x-\mu)], \tag{2.3.5}$$

where $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_n)$ is the mean vector; $\boldsymbol{\Sigma}$ is the covariance $(n \times n)$ matrix

$$\boldsymbol{\Sigma} = \begin{Vmatrix} \sigma_{11} & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{21} & \sigma_{22} & \cdots & \sigma_{2n} \\ \vdots & \vdots & & \vdots \\ \sigma_{n1} & \sigma_{n2} & \cdots & \sigma_{nn} \end{Vmatrix}; \tag{2.3.6}$$

prime denotes the transpose operator; $|\boldsymbol{\Sigma}|$ denotes the determinant of $\boldsymbol{\Sigma}$, and $\boldsymbol{\Sigma}^{-1}$ denotes the inverse matrix of $\boldsymbol{\Sigma}$.

Since $\boldsymbol{\Sigma}$ is positive definite and symmetric, there exists a unique lower triangular matrix,

$$C = \begin{Vmatrix} C_{11} & 0 & \cdots & 0 \\ C_{21} & C_{22} & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nn} \end{Vmatrix}, \tag{2.3.7}$$

such that

$$\boldsymbol{\Sigma} = \boldsymbol{C}\boldsymbol{C}'. \tag{2.3.8}$$

Thus, the vector $\boldsymbol{X}$ can be represented as

$$\boldsymbol{X} = \boldsymbol{C}\boldsymbol{Z} + \boldsymbol{\mu}, \tag{2.3.9}$$

where $\boldsymbol{Z} = (Z_1, \ldots, Z_n)$ is a normal vector with zero mean and covariance matrix equal to the identity matrix; consequently, all components Z_i, $i = 1, \ldots, n$, of $\boldsymbol{Z}$ are marginally distributed according to the standard normal distribution, $N(0, 1)$.

In order to obtain $\boldsymbol{C}$ satisfying $\boldsymbol{\Sigma} = \boldsymbol{C}\boldsymbol{C}'$, the so-called *square root method* may be used. This method computes the elements of C via a set of recursive equations as follows. From (2.3.9)

$$X_1 = C_{11}Z_1 + \mu_1. \tag{2.3.10}$$

Therefore, $\text{var}[X_1] = \sigma_{11} = C_{11}^2$ and $C_{11} = \sigma_{11}^{1/2}$. Proceeding with the second component of (2.3.9), we obtain

$$X_2 = C_{21}Z_1 + C_{22}Z_2 + \mu_2 \tag{2.3.11}$$

and

$$\sigma_{22} = \text{var}[X_2] = \text{var}[C_{21}Z_1 + C_{22}Z_2] = C_{21}^2 + C_{22}^2. \tag{2.3.12}$$

Further, from (2.3.10) and (2.3.11),

$$\sigma_{12} = \mathbb{E}\{(X_1 - \mu_1)(X_2 - \mu_2)\} = \mathbb{E}\{C_{11}Z_1(C_{21}Z_1 + C_{22}Z_2)\} = C_{11}C_{21}. \tag{2.3.13}$$

Hence, from (2.3.12) and (2.3.13), and the symmetry of Σ,

$$C_{21} = \frac{\sigma_{12}}{C_{11}} = \frac{\sigma_{12}}{\sigma_{11}^{1/2}} \tag{2.3.14}$$

$$C_{22} = \left(\sigma_{22} - \frac{\sigma_{21}^2}{\sigma_{11}}\right)^{1/2}. \tag{2.3.15}$$

Generally, the C_{ij} can be found from the recursive formula

$$C_{ij} = \frac{\sigma_{ij} - \sum_{k=1}^{j-1} C_{ik}C_{jk}}{\left(\sigma_{jj} - \sum_{k=1}^{j-1} C_{jk}^2\right)^{1/2}}, \tag{2.3.16}$$

where, by convention,

$$\sum_{k=1}^{0} C_{ik}C_{jk} = 0, \qquad 1 \le j \le i \le n,$$

are empty sums that evaluate to zero.

The following algorithm describes the generation of a multinormal variate vector.

Algorithm 2.3.3 Generation of Multinormal Vectors

1. *Generate* $Z_1, \ldots, Z_n$ *as iid variates from* $N(0, 1)$, *(see Algorithm 2.4.6).*
2. *Set*

$$C_{ij} \leftarrow \frac{\sigma_{ij} - \sum_{k=1}^{j-1} C_{ik} C_{jk}}{\left(\sigma_{jj} - \sum_{k=1}^{j-1} C_{jk}^2\right)^{1/2}}.$$

3. *Return* $\boldsymbol{X} = \boldsymbol{C}\boldsymbol{Z} + \boldsymbol{\mu}$.

2.4 GENERATING CONTINUOUS VARIATES

Sections 2.4 and 2.5 present algorithms for generating variates from commonly used continuous and discrete distributions. Of the numerous algorithms available (e.g., Devroye, 1986), we have tried to select those that are reasonably efficient and relatively simple to implement.

2.4.1 Exponential Distribution

A random variable X has an exponential distribution, if its pdf is of the form

$$f(x) = \lambda e^{-\lambda x}, \quad x > 0, \lambda > 0. \tag{2.4.1}$$

An exponential distribution is denoted by $\text{Exp}(\lambda)$, where λ is called the *rate* parameter. The following algorithm generates exponential variates via the inverse-transform method.

Algorithm 2.4.1 Generation of an Exponential Variate

1. *Generate* $U \sim \mathcal{U}(0, 1)$.
2. *Return* $X = (-1/\lambda) \ln U$ *as a variate from* $\text{Exp}(\lambda)$.

There are many alternative procedures for generating variates from the exponential distribution. The interested reader is referred to Marsaglia (1961), and Rubinstein (1981).

2.4.2 Gamma Distribution

A random variable X has a gamma distribution if its pdf is of the form

$$f(x) = \frac{x^{\beta-1}\lambda^{\beta}e^{-\lambda x}}{\Gamma(\beta)}, \qquad x > 0, \beta > 0, \lambda > 0. \tag{2.4.2}$$

The gamma distribution is denoted by $G(\lambda, \beta)$, where λ and β are called the *scale* and *shape* parameters, respectively. Since the cdf for gamma distributions does not generally exist in explicit form, the inverse-transform method cannot always be applied. Alternative methods are then needed.

Assume first that β is a positive integer, say $\beta = m$. In this case, X has the well-known Erlang distribution, denoted by Erl(m, λ), which is frequently used in simulation. It can be represented as sum of iid exponential variates Y_i, that is, $X = \sum_{i=1}^{m} Y_i$, where the Y_i's are iid exponential variates, each with mean $1/\lambda$. From Algorithm 2.4.1, $Y_i = (-1/\lambda)\ln U_i$, whence

$$X = -\frac{1}{\lambda}\sum_{i=1}^{m} \ln U_i = -\frac{1}{\lambda}\ln \prod_{i=1}^{m} U_i. \tag{2.4.3}$$

Equation (2.4.3) suggests the following straightforward generation algorithm.

Algorithm 2.4.2 Generation of an Erlang Variate

1. *Generate iid variates* $U_1, \ldots, U_m$ *from* $\mathcal{U}(0, 1)$.
2. *Return* $X = (-1/\lambda)\ln \prod_{i=1}^{m} U_i$.

It is readily seen that the execution time of the algorithm is proportional to m. Since the Erlang distribution is a special case of the gamma family, one might look for alternatives when m is large. There are many efficient procedures for generating a variate from the gamma distribution, most of which are based on the acceptance-rejection method [see Tadikamalla and Johnson (1981)].

The next proposition, due to Jöhnk (1964), gives rise to an algorithm for generating a gamma variate, $G(\lambda, \beta)$, $0 < \beta < 1$, without resorting to acceptance-rejection. It requires, however, variate generation from the beta distribution.

Proposition 2.4.1. *Let Y and Z be independent random variables from the beta distribution Be(δ, $1 - \delta$) for $0 < \delta < 1$ [see (2.4.4) below]* and Exp(1) distribution, *respectively. Then the variate $X = \lambda^{-1}YZ$ has the gamma distribution $G(\lambda, \delta)$.*

Proof. The proof is left as an exercise.

Hint: To prove the proposition, consider the transformation

$$\begin{cases} V = Z \\ X = \lambda^{-1}YZ \end{cases}.$$

Next, find the Jacobian of the transformation

$$\begin{cases} Z = V \\ Y = \lambda X/Z \end{cases}$$

and the joint pdf $f_{V,X}(\cdot)$ of the random vector (V, X). Finally, compute the marginal pdf for X by integrating the joint pdf, $f_{V,X}(v, x)$, with respect to v.

Algorithm 2.4.3 Generation of a Gamma Variate

1. *Generate two independent variates, Y and Z, from* Be$(\delta, 1-\delta)$, $0 < \delta < 1$ *(see below)*, and Exp(1), *respectively.*
2. *Return* $X = \lambda^{-1}YZ$ *as a variate from* $G(\lambda, \delta)$.

To generate a gamma variate from $G(\lambda, \beta)$ with $\beta > 1$, use the fact that the gamma family is closed under addition (of independent gamma variates) to represent β as $\beta = m + \delta$, where m is a positive integer and $0 < \delta < 1$, and write

$$X = \lambda^{-1}(W + YZ),$$

where $W \sim \text{Erl}(m, 1)$, and Y and Z are as in Algorithm 2.4.3. It follows that $X \sim G(\lambda, \beta)$.

2.4.3 Beta Distribution

A random variable X has the beta distribution if its pdf is of the form

$$f(x) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}, \qquad 0 \le x \le 1, \alpha > 0, \beta > 0. \tag{2.4.4}$$

A beta distribution is denoted by Be(α, β). Note that when $\alpha = 1$ and $\beta = 1$, the beta density Be(1, 1) coincides with the $\mathcal{U}(0, 1)$ density.

Consider first the case when either α or β equals 1. For example, for $\beta = 1$, the Be$(\alpha, 1)$ pdf is

$$f(x) = \alpha x^{\alpha-1}, \qquad 0 \le x \le 1,$$

and the corresponding cdf becomes

$$F(x) = x^{\alpha}, \qquad 0 \le x \le 1.$$

Thus, a variate $X \sim \text{Be}(\alpha, 1)$ can be generated by the inverse-transform method.

A general procedure for generating a variate from Be(α, β) is based on the fact that if $Y_1 \sim G(\alpha, 1)$, $Y_2 \sim G(\beta, 1)$, and Y_1 and Y_2 are independent, then

$$X = \frac{Y_1}{Y_1 + Y_2}$$

is distributed Be(α, β). The reader is requested to prove this assertion. The corresponding algorithm is as follows.

Algorithm 2.2.4 Generation of a Beta Variate, General Case

1. *Generate independent variates,* $Y_1 \sim G(\alpha, 1)$ *and* $Y_2 \sim G(\beta, 1)$.
2. *Return* $X = Y_1/(Y_1 + Y_2)$ *as a variate from Be*(α, β).

For integer α and β, another method may be used, based on the theory of order statistics. Let $U_1, \ldots, U_{\alpha+\beta-1}$ be random variables from $\mathcal{U}(0, 1)$. Then the αth-order statistic, $U_{(\alpha)}$, has distribution Be(α, β), (see, e.g., Wilks, 1962). The resulting algorithm is extremely simple.

Algorithm 2.4.5 Generation of a Beta Variate with Integer Parameters α and β

1. *Generate* $(\alpha + \beta - 1)$ *iid variates* $U_1, \ldots, U_{\alpha+\beta-1}$ *from* $\mathcal{U}(0, 1)$.
2. *Return the* α*th-order statistic,* $U_{(\alpha)}$, *as a variate from Be*(α, β).

It can be shown that the total number of comparisons needed to find $U_{(\alpha)}$ is $(\alpha/2)(\alpha + 2\beta - 1)$, so that this procedure loses efficiency for large α and β.

2.4.4 Normal Distribution

A random variable X has a normal distribution if its pdf is of the form

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right], \qquad -\infty < x < \infty. \tag{2.4.5}$$

The normal distribution is denoted by $N(\mu, \sigma^2)$, where μ is the mean and σ^2 is the variance.

Since the normal cdf does not have an explicit representation, the inverse-transform method cannot be applied, and some other procedures must be devised instead. We consider only generation from $N(0, 1)$ (standard normal variates) since any random variable $X \sim N(\mu, \sigma^2)$ can be represented as $X = \mu + \sigma Z$, where Z is from $N(0, 1)$. One of the earliest methods for generating variates from $N(0, 1)$ is

due to Box and Muller (1958). It is based on the fact that if U_1 and U_2 are independent random variables from $\mathcal{U}(0, 1)$, then the random variables

$$\begin{aligned} Z_1 &= (-2 \ln U_1)^{1/2} \cos(2\pi U_2) \\ Z_2 &= (-2 \ln U_1)^{1/2} \sin(2\pi U_2) \end{aligned} \tag{2.4.6}$$

are independent, from the standard normal distribution. To see that, rewrite the system (2.4.6) as

$$\begin{aligned} Z_1 &= (2V)^{1/2} \cos(2\pi U) \\ Z_2 &= (2V)^{1/2} \sin(2\pi U), \end{aligned} \tag{2.4.7}$$

where $V \sim \text{Exp}(1)$ and $U_2 = U$. It follows from (2.4.7) that

$$Z_1^2 + Z_2^2 = 2V \quad \text{and} \quad \frac{Z_2}{Z_1} = \tan(2\pi U).$$

The Jacobian of the transformation $(U, V) \to (Z_1, Z_2)$ is

$$J = \begin{vmatrix} \dfrac{\partial u}{\partial z_1} & \dfrac{\partial u}{\partial z_2} \\ \dfrac{\partial v}{\partial z_1} & \dfrac{\partial v}{\partial z_2} \end{vmatrix} = \begin{vmatrix} \dfrac{-z_2 \cos^2 2\pi u}{2\pi z_1^2} & \dfrac{\cos^2 2\pi u}{2\pi z_1} \\ z_1 & z_2 \end{vmatrix}$$

$$= \begin{vmatrix} \dfrac{-z_2}{4\pi v} & \dfrac{z_1}{4\pi v} \\ z_1 & z_2 \end{vmatrix} = -\frac{1}{4\pi v}(z_2^2 + z_1^2) = -\frac{1}{2\pi}$$

and

$$f_{Z_1,Z_2}(z_1, z_2) = f_{U,V}(u, v)|J| = \frac{1}{2\pi} \exp\left(-\frac{z_1^2 + z_2^2}{2}\right). \tag{2.4.8}$$

The last formula represents the joint pdf of two independent standard normal variates.

Algorithm 2.4.6 Generation of a Normal Variate, Box and Muller Approach

1. *Generate two independent variates, U_1 and U_2, from $\mathcal{U}(0, 1)$.*
2. *Return two independent standard normal variates, Z_1 and Z_2, by substituting U_1 and U_2 into the system of Eqs. (2.4.6).*

An alternative generation method from $N(0, 1)$ is based on the acceptance-rejection method. First, note that in order to generate a variate Y from $N(0, 1)$, one can first generate a nonnegative variate X from the pdf

$$f(x) = \sqrt{\frac{2}{\pi}} e^{-x^2/2}, \qquad x \geq 0, \tag{2.4.9}$$

and then assign to X a random sign. The validity of this procedure follows from the symmetry of the standard normal distribution about zero.

To generate a variate X from (2.4.9), recall Algorithm 2.2.5 and write $f(x)$ as

$$f(x) = Ch(x)g(x),$$

where

$$C = \left(\frac{2e}{\pi}\right)^{1/2} \tag{2.4.10}$$

$$h(x) = e^{-x} \tag{2.4.11}$$

$$g(x) = \exp\left[-\frac{(x-1)^2}{2}\right]. \tag{2.4.12}$$

The efficiency of this method is $\sqrt{\pi/2e} \simeq 0.76$.

The acceptance condition, $U \leq g(Y)$, can be written as

$$U \leq \exp[-(Y-1)^2/2], \tag{2.4.13}$$

which is equivalent to

$$-\ln U \geq \frac{(Y-1)^2}{2}, \tag{2.4.14}$$

where Y is from Exp(1). Since $-\ln U$ is also from Exp(1), the last inequality can be written as

$$V_1 \geq \frac{(V_2-1)^2}{2}, \tag{2.4.15}$$

where both $V_1 = -\ln U$ and $V_2 = Y$ are from Exp(1).

Algorithm 2.4.7 Generation of a Normal Variate, Acceptance-Rejection Approach

1. *Generate independent variates,* V_1 *and* V_2, *from* Exp(1).

2. *If* $V_1 < (V_2 - 1)^2/2$, *go to step 1.*
3. *Generate* U_1 *from* $\mathcal{U}(0, 1)$.
4. *If* $U_1 \leq 0.5$, *return* $Z = V_1$; *otherwise return* $Z = -V_1$.

2.5 GENERATING DISCRETE VARIATES

The most common procedure for generating discrete variates is the inverse-transform method [see (2.2.1) and Example 2.2.3]. An alternative approach for generating arbitrary discrete variates with a *finite* range of values is the so-called *alias method*, due to Walker (1977). The alias method is based on the fact that any n-point pdf

$$P(X = x_i) = p_i, \qquad i = 1, \ldots, n$$

can be represented as an equally weighted mixture of $n - 1$ discrete pdf's, $g^{(k)}$, $k = 1, \ldots, n - 1$, each having at most *two* nonzero components. That is, any n-point pdf f can be represented as

$$f = \frac{1}{n-1} \sum_{k=1}^{n-1} g^{(k)} \tag{2.5.1}$$

for suitably defined 2-point pdf's $g^{(k)}$, $k = 1, \ldots, n$ (see Walker, 1977).

The alias method is rather general and efficient but requires some initial setup and extra storage for the $n - 1$ pdf's, $g^{(k)}$.

Lemma 2.5.1. *For any n-point pdf* $f(x_i)$, $i = 1, \ldots, n$:

(a) There exists an index i, such that $p_i < 1/(n - 1)$; $i = 1, \ldots, n$.
(b) Given this index i, there exists an index $j, j \neq i$, *such that* $p_i + p_j \geq 1/(n - 1)$.

Proof. The proof of this lemma is left as an exercise.

With Lemma 2.5.1 in hand, a procedure for computing these simple pdf's, $g^{(k)}$, $k = 1, \ldots, n - 1$, can be devised such that (2.5.1) holds for an arbitrary n-point pdf, f. For more details, see Walker (1977) or Law and Kelton (1991).

The rest of this section presents several algorithms for generating variates from commonly used discrete distributions.

2.5.1 Bernoulli Distribution

A random variable X has a Bernoulli distribution, if its pdf is of the form

$$f(x) = p^x(1 - p)^{1-x}, \qquad x = 0, 1, \quad 0 < p < 1. \tag{2.5.2}$$

A Bernoulli distribution is denoted by Ber(p), where p is the probability of success. A Bernoulli variate returns the outcome of a binary trial (0 = failure, 1 = success). Applying the inverse-transform method, one readily obtains the following generation algorithm.

Algorithm 2.5.1 Generation of a Bernoulli Variate

1. *Generate* $U \sim \mathcal{U}(0, 1)$.
2. *If* $U \leq p$, *return* $X = 1$; *otherwise return* $X = 0$.

2.5.2 Binomial Distribution

A random variable has a binomial distribution if its pdf is of the form

$$f(x) = \binom{n}{x} p^x (1-p)^{n-x}, \qquad x = 0, 1, \ldots, n, \quad 0 < p < 1. \tag{2.5.3}$$

A binomial distribution is denoted by Bin(n, p), and the corresponding binomial variate is the sum of n iid Bernoulli trials, each with parameter p. The simplest generation algorithm can be written as follows.

Algorithm 2.5.2 Generation of a Binomial Variate

1. *Generate iid variates* $Y_1, \ldots, Y_n$ *from* Ber(p).
2. *Return* $X = \sum_{i=1}^{n} Y_i$ *as a variate from* Bin(n, p).

Since the execution time of Algorithm 2.5.2 is proportional to n, one is motivated to use alternative methods for large n. For example, one may consider the normal distribution as an approximation to the binomial. In particular, as n increases, the distribution of

$$Z = \frac{X - np + \frac{1}{2}}{[np(1-p)]^{1/2}} \tag{2.5.4}$$

approaches $N(0, 1)$.

To obtain a binomial variate, we generate Z from $N(0, 1)$, solve (2.5.4) for X, and truncate to the nearest nonnegative integer, that is,

$$X = \max\{0, \lfloor -0.5 + np + Z(np(1-p))^{1/2} \rfloor\}, \tag{2.5.5}$$

where $\lfloor \alpha \rfloor$ denotes the integer part of α. One should consider replacing the binomial with the approximating normal variate for $np > 10$ with $p \geq \frac{1}{2}$, and for $n(1-p) > 10$ with $p < \frac{1}{2}$.

It is worthwhile to note that if $Y \sim \text{Bin}(n, p)$, then $n - Y \sim \text{Bin}(n, 1-p)$. Hence, to enhance efficiency, one may elect to generate X from $\text{Bin}(n, p)$ according to

$$X = \begin{cases} Y_1 \sim \text{Bin}(n, p) & \text{if } p \le \frac{1}{2} \\ n - Y_2,\ Y_2 \sim \text{Bin}(n, 1-p) & \text{if } p > \frac{1}{2}. \end{cases}$$

2.5.3 Geometric Distribution

A random variable has a geometric distribution, if its pdf is of the form

$$f(x) = p(1-p)^x, \qquad x = 0, 1, \ldots, \quad 0 < p < 1. \tag{2.5.6}$$

A geometric distribution is denoted by $\text{Ge}(p)$, and the corresponding geometric variate is the number of trials separating successes in an iid sequence of Bernoulli trials.

We now present an algorithm that is based on the relationship between the exponential and geometric distributions. Let $Y \sim \text{Exp}(\beta)$, whence

$$P(x < Y \le x+1) = \frac{1}{\beta}\int_x^{x+1} e^{-y/\beta}dy = e^{-x/\beta}(1 - e^{-1/\beta}). \tag{2.5.7}$$

Thus for integer $x \ge 0$, Eq. (2.5.7) is the geometric distribution $\text{Ge}(p = 1 - e^{-1/\beta})$.

In particular, for $\beta = -1/\ln(1-p)$, Eq. (2.5.7) is identical to (2.5.6). Therefore, since $P(Y \le x) = 1 - e^{-x/\beta}$,

$$X = \left\lfloor \frac{\ln U}{\ln(1-p)} \right\rfloor = \left\lfloor -\frac{V}{\ln(1-p)} \right\rfloor, \tag{2.5.8}$$

where $V = -\ln(U)$ is a standard exponential variate implying that X is from $\text{Ge}(p)$. Hence, to generate a variate from $\text{Ge}(p)$, we first generate a variate from the exponential distribution with $\beta = -1/\ln(1-p)$ and then truncate the obtained value to the nearest integer.

Algorithm 2.5.3 Generation of a Geometric Variate

1. *Generate* $V \sim \text{Exp}[-1/\ln(1-p)]$.
2. *Return* $X = \lfloor -V/\ln(1-p) \rfloor$ *as a variate from* $\text{Ge}(p)$.

2.5.4 Poisson Distribution

A random variable X has a Poisson distribution, if its pdf is of the form

$$f(n) = \frac{\lambda^n e^{-\lambda}}{n!}, \qquad n = 0, 1, \ldots; \quad \lambda > 0. \tag{2.5.9}$$

A Poisson distribution is denoted by $P(\lambda)$, where λ is the *rate* parameter. A Poisson variate, X, is the maximal number of iid exponential variates (with common parameter λ) whose sum does not exceed one.

The following algorithm is based on the relationship between the $P(\lambda)$ and $\text{Exp}(\lambda)$ distributions. Write

$$\sum_{i=0}^{i} Y_j \le 1 \le \sum_{i=0}^{i+1} Y_j, \tag{2.5.10}$$

where Y_j, $j = 0, 1, \ldots, i+1$, are iid variates from $\text{Exp}(\lambda)$. Let the Y_j be viewed as intervals, laid end to end, and let the end point of each interval be interpreted as an "event"(e.g., a customer arrival, say, at a queue). Then in this setting Eq. (2.5.10) means that the number of events by time 1, denoted $X(1)$, is Poisson distributed (with mean λ) and can be written as

$$X(1) = \max\left\{n: \sum_{j=1}^{n} Y_j \le 1\right\}. \tag{2.5.11}$$

Further, since $Y_j = (-1/\lambda)\ln U_n$, we can rewrite (2.5.11) as

$$\begin{aligned} X &= \max\left\{n: \sum_{j=1}^{n} -\ln U_j \le \lambda\right\} \\ &= \max\left\{n: \ln\left(\prod_{j=1}^{n} U_j\right) \ge -\lambda\right\} \\ &= \max\left\{n: \prod_{j=1}^{n} U_j \ge e^{-\lambda}\right\}. \end{aligned} \tag{2.5.12}$$

To see that the random variable X has the desired (Poisson) distribution, note again that $-\ln U_j$ is exponential with rate 1. Interpreting $-\ln U_j$, $j \ge 1$, as the interarrival times of a Poisson process with rate 1, we have that X is equal to the number of events by time λ. Thus, X is Poisson distributed with mean λ.

Algorithm 2.5.4 Generating a Poisson Variate

1. *Set* $n \leftarrow 1$ *and* $a \leftarrow 1$.
2. *Generate an independent* $U_n \sim \mathcal{U}(0, 1)$ *and set* $a \leftarrow aU_n$.
3. *If* $a \ge e^{-\lambda}$, *then set* $n \leftarrow n + 1$ *and go to step 2.*
4. *Otherwise, return* $X = n - 1$, *as a variate from* $P(\lambda)$.

It is readily seen that for large λ, this algorithm becomes slow ($e^{-\lambda}$ is small for large λ, and more random numbers, U_j, are required to satisfy $\prod_{j=1}^{n} U_j < e^{-\lambda}$).

Alternative approaches use the inverse-transform method with an efficient search (see Atkinson, 1979a,b, and Devroye, 1981), or the alias method.

2.6 GENERATING POISSON PROCESSES

The rest of this chapter is concerned with generating variate sequences. For example, in order to simulate a renewal process over the interval $[0, t]$, one generates a sequence of iid variates of random length

$$N(t) = \min\left\{n: \sum_{i=1}^{n} X_i > t\right\}. \tag{2.6.1}$$

Clearly, if X_i, $i = 1, 2, \ldots$, represent the times between arrivals, then (2.6.1) gives rise to $N - 1$ events which occur at times $X_1, X_1 + X_2, \ldots, X_1 + \cdots + X_{N(t)-1}$, within the interval $[0, t]$.

This section treats the generation of stationary and nonstationary Poisson processes.

2.6.1 Generating a Stationary Poisson Process

In the special case where $X_i \sim \text{Exp}(\lambda)$, the process $\{N(t)\}$ is a stationary Poisson process with rate λ. Thus, the $X_i = (-1/\lambda)\ln U_i$, $i = 1, \ldots, n$, are the exponentially distributed time intervals between the $(i-1)$st, and the ith arrival events, and the time point of the jth arrival is $A_n = \sum_{i=1}^{j} X_i$.

The algorithm for generating a Poisson process $\{N(t)\}$ over the interval $[0, T]$ is given below.

Algorithm 2.6.1 Generation of a Stationary Poisson Process

1. *Set* $t \leftarrow 0$ *and* $n \leftarrow 0$.
2. *Set* $n \leftarrow n + 1$.
3. *Generate an independent variate* $U_n \sim \mathcal{U}(0, 1)$.
4. *Set* $t \leftarrow t - (1/\lambda)\ln U_n$ *and declare an arrival event.*
5. *If* $t > T$, *stop*; *otherwise go to step 2.*

Note that t is the variable recording the time of the most recent arrival event (by convention, $t = 0$ is a fictitious arrival time).

2.6.2 Generating a Nonstationary Poisson Process

Thinning is a simple method, due to Lewis and Shedler (1979), for generating nonstationary Poisson processes with rate function $\lambda(t)$. For an alternative method, see Law and Kelton (1991) and Ross (1990).

The method first generates a stationary Poisson process with rate $\lambda = \max_{0 \le t \le T} \lambda(t)$, which is assumed to be finite. Each arrival event of the stationary Poisson process is rejected (thinned) with probability $1 - \lambda(A_n)/\lambda$, where A_n is the (random) arrival time of the nth event. The surviving events define the desired nonstationary Poisson process.

The algorithm for generating a non-stationary Poisson process $\{N(t)\}$ over the interval $[0, T]$ is similar to the previous algorithm.

Algorithm 2.6.2 Generating a Nonstationary Poisson Process

1. *Set* $t \leftarrow 0$ *and* $n \leftarrow 0$.
2. *Set* $n \leftarrow n + 1$.
3. *Generate an independent variate* $U_n \sim \mathcal{U}(0, 1)$.
4. *Set* $t \leftarrow t - (1/\lambda) \ln U_n$.
5. *If* $t > T$, *stop*; *otherwise, continue.*
6. *Generate an independent variate* $V_n \sim \mathcal{U}(0, 1)$.
7. *If* $V_n \le \lambda(t)/\lambda$, *declare an arrival event and go to step 2.*

*2.7 GENERATING RANDOM VECTORS, UNIFORMLY DISTRIBUTED OVER VARIOUS REGIONS

This section deals with several algorithms for generating random vectors, uniformly distributed over the surface and interior of (a) a simplex, (b) a hypersphere, and (c) a hyperellipsoid. Most of the material is taken from Rubinstein (1982). For a landmark paper on generating vectors, uniformly distributed over bounded regions, see Smith (1984); and for efficient generation methods of random vectors over the surface and interior of open regions with arbitrary distributions, see Borovkov (1994) and Claude et al. (1993).

2.7.1 Generating Random Vectors over a Simplex

The pdf of a random vector $\boldsymbol{X}$, uniformly distributed over the interior of an n-dimensional simplex,

$$S_n = \left\{(x_1, \ldots, x_n): x_i \ge 0, i = 1, \ldots, n, \sum_{i=1}^{n} x_i \le 1\right\}, \tag{2.7.1}$$

is

$$f(x) = \begin{cases} n! & \text{if } x = (x_1, \ldots, x_n) \in S_n, \\ 0 & \text{otherwise.} \end{cases} \tag{2.7.2}$$

This density is a special case of the density

$$f(x) = \frac{\Gamma(v_1 + \cdots + v_{n+1})}{\Gamma(v_1) \cdots \Gamma(v_{n+1})} x_1^{v_1 - 1} \cdots x_n^{v_n - 1} (1 - x_1 - \cdots - x_n)^{v_{n+1} - 1}, \tag{2.7.3}$$

with parameters $v_1, \ldots, v_n, v_{n+1}$. A density of the form (2.7.3) is called a *Dirichlet density*, and the corresponding *Dirichlet distribution* is denoted by $D(v_1, \ldots, v_n, v_{n+1})$ (see Wilks, 1962). Indeed, for $v_1 = v_n = v_{n+1} = 1$, Eq. (2.7.3) reduces to Eq. (2.7.2), and the corresponding distribution is denoted by $D(1, \ldots, 1, 1)$. It is also known (see, e.g., Wilks, 1962) that if Y_i, $i = 1, \ldots, n+1$, are independent random variables from gamma distributions with shape parameter $v_i > 0$ and scale parameter 1, namely,

$$f_{Y_i}(y) = \begin{cases} \dfrac{y^{v_i - 1} e^{-y}}{\Gamma(v_i)} & y > 0 \\ 0 & \text{otherwise,} \end{cases} \tag{2.7.4}$$

then the (normalized) random vector,

$$\boldsymbol{X} = (X_1, \ldots, X_n) = \left(\frac{Y_1}{\sum_{i=1}^{n+1} Y_i}, \ldots, \frac{Y_n}{\sum_{i=1}^{n+1} Y_i} \right), \tag{2.7.5}$$

is distributed $D(v_1, \ldots, v_n, v_{n+1})$. Observing that for the special case $v_i = 1$, $i = 1, \ldots, n+1$, each Y_i is distributed Exp(1), the algorithm for generating the desired random vector $\boldsymbol{X}$ can be written as follows:

Algorithm 2.7.1 Generating a Random Vector over a Simplex (Exponential Approach)

1. *Generate* $n+1$ *variates* $Y_1, \ldots, Y_n, Y_{n+1}$, *each from* Exp(1).
2. *Apply formula (2.7.5) and return the vector* $(X_1, \ldots, X_n)$.

It is known (e.g., Rubinstein, 1982) that if we increase the dimension in (2.7.5) from n to $n+1$, namely,

$$\boldsymbol{X} = (X_1, \ldots, X_{n+1}) = \left(\frac{Y_1}{\sum_{i=1}^{n+1} Y_i}, \ldots, \frac{Y_{n+1}}{\sum_{i=1}^{n+1} Y_i} \right), \tag{2.7.6}$$

then the resultant $\boldsymbol{X} = (X_1, \ldots, X_{n+1})$ is uniformly distributed over the surface of the $(n+1)$-dimensional simplex,

$$S_{n+1} = \left\{(x_1, \ldots, x_{n+1}) : x_i \geq 0, i = 1, \ldots, n+1, \sum_{i=1}^{n+1} x_k = 1\right\}, \tag{2.7.3}$$

whereas the original vector, $\boldsymbol{X} = (X_1, \ldots, X_n)$, is uniformly distributed over the interior of the n-dimensional simplex considered in (2.7.5).

An alternative procedure for generating random vectors, uniformly distributed over the interior of an n-dimensional simplex, is based on the following result. Let $Y_{(1)}, \ldots, Y_{(n)}$ be the order statistics where the variates $Y, \ldots, Y_n$ are iid with a continuous common cdf $F_Y(y)$. Then the random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$, defined by

$$\begin{aligned} X_1 &= F_Y(Y_{(1)}), \\ X_2 &= F_Y(Y_{(2)}) - F_Y(Y_{(1)}), \\ &\vdots \\ X_n &= F_Y(Y_{(n)}) - F_Y(Y_{(n-1)}), \end{aligned} \tag{2.7.4}$$

is distributed $D(1, \ldots, 1, 1)$ (see Wilks, 1962). Observe that in the special case that the random variables $Y_j = U_j$ are distributed $\mathcal{U}(0, 1)$, Eq. (2.7.7) can be rewritten as

$$\begin{aligned} X_1 &= U_{(1)}, \\ X_2 &= U_{(2)} - U_{(1)}, \\ &\vdots \\ X_n &= U_{(n)} - U_{(n-1)}. \end{aligned} \tag{2.7.5}$$

The following is an alternative algorithm for generating a random vector from $D(1, \ldots, 1, 1)$.

Algorithm 2.7.2 Generating a Vector over a Simplex (Order Statistics Approach)

1. *Generate n independent variates* $U_1, \ldots, U_n$ *from* $\mathcal{U}(0, 1)$.
2. *Permute* $U_1, \ldots, U_n$ *into the order statistics* $U_{(1)}, \ldots, U_{(n)}$.
3. *Apply formula (2.7.8), and return the vector* $\boldsymbol{X} = (X_1, \ldots, X_n)$.

Again, if we increase the dimension in (2.7.8) from n to $n+1$, namely,

$$\begin{aligned} X_1 &= U_{(1)}, \\ X_2 &= U_{(2)} - U_{(1)}, \\ &\vdots \\ X_n &= U_{(n)} - U_{(n-1)}, \\ X_{n+1} &= 1 - U_{(n)}, \end{aligned} \tag{2.7.9}$$

then the resulting vector, $\boldsymbol{X} = (X_1, \ldots, X_{n+1})$, is uniformly distributed over the surface of the $(n+1)$-dimensional simplex.

Note that both Algorithms 2.7.1 and 2.7.2 are suitable for generating random vectors, uniformly distributed over an n-dimensional simplex with vertices

$$\boldsymbol{x}_0 = (0, 0, \ldots, 0), \boldsymbol{x}_1 = (1, 0, \ldots, 0), \ldots, \boldsymbol{x}_n = (0, 0, \ldots, 1).$$

Consider now the generation of random vectors, uniformly distributed over an n-dimensional simplex with arbitrary vertices, say, $\boldsymbol{z}_0, \boldsymbol{z}_1, \ldots, \boldsymbol{z}_n$, where $\boldsymbol{z}_i = (z_{i1}, \ldots, z_{in})$, $i = 0, 1, \ldots, n$. It is readily seen that if $\boldsymbol{X}$ is uniformly distributed over the simplex (2.7.1), then the linear transformation

$$\boldsymbol{Z} = \boldsymbol{C}\boldsymbol{X} + \boldsymbol{z}_0$$

with the matrix

$$\boldsymbol{C} = \begin{bmatrix} z_{11} - z_{01} & z_{12} - z_{02}, \ldots, & z_{1n} - z_{0n} \\ z_{21} - z_{01} & z_{22} - z_{02}, \ldots, & z_{2n} - z_{0n} \\ \vdots & \vdots & \vdots \\ z_{n1} - z_{01} & z_{n2} - z_{02}, \ldots, & z_{nn} - z_{0n} \end{bmatrix}$$

preserves uniformity, that is, the vector $\boldsymbol{Z}$ will be uniformly distributed on the simplex with vertices $\boldsymbol{z}_0, \boldsymbol{z}_1, \ldots, \boldsymbol{z}_n$.

2.7.2 Generating Random Vectors, Uniformly Distributed over a Unit Hypersphere

The density function of a random vector, uniformly distributed over the surface of an n-dimensional sphere of radius r, is

$$f_Y(y) = \begin{cases} \dfrac{\Gamma(n/2)}{2\pi^{n/2} r^{n-1}} & y = \left\{(y_1, \ldots, y_n): \sum_{i=1}^{n} y_i^2 = r^2\right\}, \\ 0 & \text{otherwise.} \end{cases} \tag{2.7.10}$$

Formula (2.7.10) follows from the fact that the surface "size" of the n-dimensional sphere of radius r is

$$\frac{2\pi^{n/2}r^{n-1}}{\Gamma(n/2)}.$$

Before proceeding with the algorithms for generating points, uniformly distributed over the surface and interior of the n-dimensional sphere, it should be noted that any *surface* algorithm (see below) can be converted into an *interior* algorithm and vice versa. Indeed, let $\boldsymbol{Y} = (Y_1, \ldots, Y_n)$ be a vector, uniformly distributed over the surface of an n-dimensional sphere, and let $U \sim \mathcal{U}(0, 1)$. Then the vector $\boldsymbol{X} = (X_1, \ldots, X_n)$, with components $X_i = U^{1/n}Y_i$, is uniformly distributed over the interior of that hypersphere. Conversely, if $\boldsymbol{X} = (X_1, \ldots, X_n)$ is a vector, uniformly distributed over the interior of the n-dimensional sphere, then the vector $\boldsymbol{Y} = (Y_1, \ldots, Y_n)$ with components $Y_i = (\sum_{i=1}^n X_i^2)^{-1/2}X_i$, is distributed uniformly over the surface of that hypersphere. This holds, since $\boldsymbol{Y}$ is the projection of interior sphere points onto the surface of the sphere.

The algorithm for generating a vector, uniformly distributed over the surface of an n-dimensional unit sphere, is based on von Neumann's acceptance-rejection method.

Algorithm 2.7.3 Generating Random Vectors over the Surface of a Unit Hypersphere (Acceptance-Rejection Approach)

1. *Generate iid variates* $U_i \sim \mathcal{U}(0, 1)$, $i = 1, \ldots, n$.
2. *Set* $Z_i \leftarrow 1 - 2U_i$, $i = 1, \ldots, n$, *and* $Y^2 \leftarrow \sum_{i=1}^n Z_i^2$.
3. *If* $Y^2 \geq 1$, *then go to step 1.*
4. *Otherwise, if* $Y^2 < 1$, *return* $\boldsymbol{X} = (X_1, \ldots, X_n)$, *where* $X_i = Z_i/Y$, $i = 1, \ldots, n$.

To obtain a point, uniformly distributed over the interior of an n-dimensional unit sphere, we need only replace step 4 in Algorithm 2.7.3 by

4′. *Otherwise, if* $Y^2 < 1$, *return* $\boldsymbol{Y} = (Y_1, \ldots, Y_n)$, *all other steps remaining the same.*

The main advantage of the acceptance-rejection method is its simplicity. Its main disadvantage is that the number of trials needed to generate a hypersphere point increases explosively in n (see Section 4.2). For this reason, it can be recommended only for low dimensions ($n \leq 5$).

Another algorithm for generating a vector, uniformly distributed over the surface of a unit hypersphere, is based on the following result.

Theorem 2.7.1. *Let* $X_1, \ldots, X_n$ *be iid variates from* $N(0, 1)$, *and define* $Z = (\sum_{i=1}^n X_i^2)^{1/2} = V$. *Then the vector*

$$\boldsymbol{Y} = (Y_1, \ldots, Y_n) = \left(\frac{X_1}{Z}, \ldots, \frac{X_n}{Z}\right) \tag{2.7.11}$$

is distributed uniformly over the surface of the unit hypersphere $\{(y_1, \ldots, y_n): \sum_{i=1}^n y_i^2 = 1\}$.

Proof. Consider the transformation $(x_1, \ldots, x_n, z) \to (y_1, \ldots, y_n, v)$, given by

$$\begin{cases} y_i(x_1, \ldots, x_n, z) = \dfrac{x_i}{z}, & i = 1, \ldots, n \\ v(x_1, \ldots, x_n, z) = z. \end{cases} \tag{2.7.12}$$

Its Jacobian is $J = v^n$, since the inverse transformation is

$$\begin{cases} x_i(y_1, \ldots, y_n, v) = vy_i, & i = 1, \ldots, n \\ z(y_1, \ldots, y_n, v) = v. \end{cases}$$

But

$$f_{X_1,\ldots,X_n,Z}(x_1, \ldots, x_n, z) = \frac{1}{(2\pi)^{n/2}} e^{-(1/2)z^2} I\left(z, \left(\sum x_i^2\right)^{1/2}\right),$$

where

$$I(x, y) = \begin{cases} 1, & \text{if } x = y \\ 0, & \text{otherwise.} \end{cases}$$

This implies

$$f_{Y_1,\ldots,Y_n,V}(y_1, \ldots, y_n, v) = v^n \frac{1}{(2\pi)^{n/2}} \exp\left(-\sum_{i=1}^n \frac{(y_i v)^2}{2}\right) I\left(v, \left(\sum_{i=1}^n (y_i v)^2\right)^{1/2}\right). \tag{2.7.13}$$

In view of

$$\sum_{i=1}^n y_i^2 = \sum_{i=1}^n \left(\frac{x_i}{z^2}\right)^2 = \frac{1}{z^2} \sum_{i=1}^n x_i^2 = 1,$$

we have $I(v, (\sum_{i=1}^n (y_i v)^2)^{1/2}) = 1$, so that (2.7.13) can be further simplified to

$$\begin{aligned} f_{Y_1,\ldots,Y_n,V}(y_1, \ldots, y_n, v) &= v^n \frac{1}{(2\pi)^{n/2}} \exp\left(-\frac{v^2}{2} \sum_{i=1}^n y_i^2\right) \\ &= v^n \frac{1}{(2\pi)^{n/2}} e^{-v^2/2}. \end{aligned} \tag{2.7.14}$$

It follows from (2.7.13) and (2.7.14) that $(Y_1, \ldots, Y_n)$ and V are independent, and that the vector $(Y_1, \ldots, Y_n)$ is uniformly distributed over the surface of the unit sphere, $\{(y_1, \ldots, y_n)\colon \sum_{i=1}^n y_i^2 = 1\}$.

Theorem 2.7.1 motivates the following alternative generation algorithm.

Algorithm 2.7.4 Generating Random Vectors over the Surface of a Unit Hypersphere [Box and Muller (1958)]

1. *Generate n iid variates* X_i, $i = 1, \ldots, n$, *from* $N(0, 1)$.
2. *Compute* $Z = (\sum_{i=1}^n X_i^2)^{1/2}$.
3. *Return* $\mathbf{Y} = (Y_1, \ldots, Y_n)$, *where* $Y_i = X_i/Z$, $i = 1, \ldots, n$.

An alternative recursive procedure is as follows. First select a point on the surface of the one-dimensional sphere ($x_1 = 1$ or $x_1 = -1$), and then continue recursively until the required dimensionality is reached: Given that a point $(x_1, x_2, \ldots, x_n)$ on the surface of an n-dimensional sphere has already been generated, the point on the $(n+1)$-dimensional sphere is selected as

$$(Rx_1, Rx_2, \ldots, Rx_n, S\sqrt{1 - R^2}),$$

where S assumes the values $+1$ and -1 with probability $\frac{1}{2}$ each, and R has the pdf

$$f_R(r) = \begin{cases} k\dfrac{r^{n-1}}{\sqrt{1-r^2}} & 0 \le r \le 1, k = \left(\displaystyle\int_0^1 \frac{r^{n-1}}{\sqrt{1-r^2}}\,dr\right)^{-1}, \\ 0 & \text{otherwise.} \end{cases}$$

Consider now the polar transformation

$$\begin{aligned} X_1 &= R\cos\varphi_1 \cos\varphi_2 \cdots \cos\varphi_{n-2}\cos\varphi_{n-1} \\ X_2 &= R\cos\varphi_1 \cos\varphi_2 \cdots \cos\varphi_{n-2}\sin\varphi_{n-1} \\ X_3 &= R\cos\varphi_1 \cos\varphi_2 \cdots \sin\varphi_{n-2} \\ &\vdots \\ X_{n-1} &= R\cos\varphi_1 \sin\varphi_2 \\ X_n &= R\sin\varphi_1 \end{aligned} \tag{2.7.15}$$

where $-\frac{1}{2}\pi \le \varphi_i \le \frac{1}{2}\pi$, $i = 1, \ldots, n-2$, $0 \le \varphi_{n-1} \le 2\pi$, and $0 \le R \le 1$ has pdf f_R; furthermore, R and $\varphi_1, \ldots, \varphi_{n-1}$ are mutually independent variates. The following algorithm generates points, uniformly distributed over the interior of the hypersphere of radius r.